Distance Comparison Operations Are Not Silver Bullets in Vector Similarity Search: A Benchmark Study on Their Merits and Limits [EA&B]

Zhuanglin Zheng, Yuxiang Zeng, Chenchen Liu, Yunzhen Chi, Binhan Yang, Yongxin Tong
SKLCCSE Lab, BDBC and IRI, Beihang University, Beijing, China
{zzlin, yxzeng, 23371020, chiyz, yangbh, yxtong}@buaa.edu.cn

Abstract*—Distance Comparison Operations (DCOs), which decide whether the distance between a data vector and a query is within a threshold, are a critical performance bottleneck in vector similarity search. Recent DCO methods that avoid full-dimensional distance computations promise significant speedups, but their readiness for production vector database systems remains an open question. To address this, we conduct a comprehensive benchmark of 8 DCO algorithms across 10 datasets (with up to 100M vectors and 12,288 dimensions) and diverse hardware configurations (CPUs with/without SIMD, and GPUs). Our study reveals that *these methods are not silver bullets*: their efficiency is highly sensitive to data dimensionality, degrades under out-of-distribution queries, and is unstable across hardware. Yet, our evaluation also demonstrates often-overlooked merits: they can accelerate index construction and data updates. Despite these benefits, their unstable performance, which can be slower than a full-dimensional scan, leads us to conclude that *recent algorithmic advancements in DCO are not yet ready for production deployment.

***Index Terms*—Vector Database, Similarity Search, Benchmark.**

I. Introduction

Vector similarity search is a fundamental component of modern data-intensive applications, including recommendation systems, search engines, and Retrieval-Augmented Generation (RAG). A central challenge in this domain is improving search efficiency. While the majority of research has focused on designing index structures, a recent and promising line of work optimizes efficiency at a more granular operator level.

These studies [1]–[6] identify the *Distance Comparison Operation (DCO)* (which determines whether the distance between a data vector and a query vector is within a given threshold) as a critical efficiency bottleneck. Traditional vector database systems [7]–[9] often perform DCOs by exhaustively computing the full-dimensional distance, a naive method denoted as Full Dimension Scanning ("FDScanning" as short). In contrast, state-of-the-art methods [1]–[4] leverage techniques like hypothesis testing or prediction to scan only partial dimensions and infer the comparison result. These approaches have been reported to improve query throughput by 2–4× over FDScanning with almost no loss in recall.

The impressive results raise a natural yet critical question: *Are these new DCO algorithms ready for deployment in production vector databases?* To answer this, we analyzed the evaluation protocols of prior research (see Table I) and identified several key limitations in their evaluations [1]–[4]:

- **Narrow Dimensionality Coverage**: They are largely confined to the dimensionality range of 128–960, neglecting lower-dimensional and, particularly, ultra-high-dimensional data common in embeddings from LLMs.
- **Insufficient Direct Comparison**: They lack empirical comparisons between the state-of-the-art DCO methods.
- **Restricted Hardware Configuration**: They typically use CPUs with SIMD disabled, which does not reflect the computing capabilities of a modern server.
- **Limited Application Scenario**: They mainly focus on in-distribution queries, overlooking practical scenarios like Out-of-Distribution (OOD) queries and index building.

These limitations prevent a definitive answer to the previous question and motivate our work. To this end, we conduct a comprehensive benchmark of 8 DCO methods across 10 datasets and diverse hardware. Our evaluation reveals that *these methods are not silver bullets*. Their effectiveness is highly contingent, and their adoption requires careful consideration. Specifically, we have the following findings:

(1) **Dimensionality Sensitivity**: Their performance is highly sensitive to data dimensionality. While they excel within a moderate range, their advantage degrades significantly outside of it, at times falling behind FDScanning by up to 20%–42%.

(2) **Robustness Dilemma**: Existing methods face a dual challenge. On the one hand, they are vulnerable to OOD queries that are common in applications like multimodal retrieval. On the other hand, their efficiency gains are unstable across hardware. For instance, enabling SIMD can reduce or even reverse their advantage over FDScanning.

(3) **Benefit Beyond Query**: An often-overlooked merit of DCO methods lies in accelerating index construction and data insertions by up to 64% and 63%, respectively.

(4) **No Universal Winner**: There is no single dominant method. Their performance ranking varies drastically with data characteristics, query distribution, and hardware.

Contribution. We make the following major contributions:

- We present the first comprehensive benchmark for evaluating DCOs in vector similarity search.
- Through extensive evaluation, we provide a detailed analysis of the merits and limitations of existing DCO methods. The in-depth analysis leads to the final answer: *the recent algorithmic advancements in DCO, while promising, are not yet ready for production deployment.*

TABLE I: Comparisons of evaluation setup in prior work on Distance Comparison Operation (DCO)

Related Work	#(Dataset)	Dimensionality	Scalability	#(Algorithm) Tested	Compared Work	Hardware Environment	Application Scenario	Distance Metric	OOD Query
[1]	6	$256 \sim 960$	5 million	3	N/A (first work)	CPU w/o SIMD	Query	Euclidean	×
[2]	6	$256 \sim 960$	5 million	3	[1]	CPU w/o SIMD	Query	Euclidean	×
[3]	8	$128 \sim 960$	100 million	5	[1]	CPU w/o SIMD	Query	Euclidean, IP	✓
[4]	6	$96 \sim 4096$	1 million	3	[1]	CPU ± SIMD	Query	Euclidean	×
Ours	**10**	$\mathbf{96 \sim 12288}$	**100 million**	**8**	[1]–[4]	**CPU ± SIMD & GPU**	**Query, Update, Index Construction**	**Euclidean, IP**	✓

SIMD: Single Instruction Multiple Data; IP: Inner Product; OOD: Out-of-Distribution, Ref. [3] uses only synthetic data for OOD evaluation.

- We derive practical guidelines for selecting the optimal DCO method under different scenarios and highlight future research directions. To foster further work, we have open-sourced the benchmark on GitHub [10].

Road Map. The rest of this paper is structured as follows. Sec. II defines the DCO problem, and Sec. III introduces existing algorithms. Next, Sec. IV and Sec. V detail the benchmark setup and analyze evaluation results. Finally, Sec. VI reviews related work and Sec. VII concludes the paper.

II. Preliminary

This section defines the *Distance Comparison Operation (DCO)* problem. Table II summarizes the major notations.

A. Basic Concepts

Before defining the DCO, we first introduce two basic concepts: vector data and vector similarity search.

Definition 1 (Vector Data): A vector data ("vector" as short) o is defined as an ordered sequence $(x_1, x_2, \cdots, x_D)$. Here, D is the vector's dimension, and x_i represents the i-th coordinate. The function $dis(o_1, o_2)$ denotes the distance between two vectors o_1 and o_2, which is a measure of their similarity.

We use $\mathcal{O}$ to denote a dataset containing N vectors, each with D dimensions. In practice, vector datasets can be classified into three kinds based on their dimensionality [11], [12]:

- **Low-Dimensional Vector Dataset**: These are defined by a moderate dimensionality ($D \in [10, 100]$). They are prevalent in classical statistics and traditional machine learning (ML) applications [13], [14].
- **High-Dimensional Vector Dataset**: These datasets exhibit a large number of dimensions ($D \in (100, 1000]$). They usually consist of embedding vectors generated by deep learning models in domains like computer vision and natural language processing [15], [16].
- **Ultra-High-Dimensional Vector Dataset**: This kind encompasses datasets with dimensionality on the order of 10^3 or higher. They often arise from raw sensor data and embedding layers of LLMs [17], [18].

Distance functions are diverse, and commonly used examples include Euclidean distance, inner product, and cosine similarity. For simplicity, we use Euclidean distance as the default distance function in the rest of the paper.

Definition 2 (Vector Similarity Search): Given a vector dataset $\mathcal{O}$, a query vector $q \in \mathbb{R}^D$, and a positive integer k, vector similarity search aims to find the k nearest neighbor (KNN) vectors $\mathcal{S} \subseteq \mathcal{O}$ to the query vector q satisfying

$$|\mathcal{S}| = k \text{ and } \forall v \in \mathcal{S}, \forall u \in (\mathcal{O} \setminus \mathcal{S}),\ dis(v, q) \leq dis(u, q)$$

TABLE II: Summary of frequently used notations

Notation	Description
$\mathcal{O}$	A dataset $\mathcal{O}$ of N vectors, each of dimensionality D
$\mathcal{S}$	Query answer
q, k	Query vector q and the number k of nearest neighbors
$dis(o_1, o_2)$	Distance between two vectors o_1 and o_2
τ, $\widetilde{dis}$	A given distance threshold and estimated distance
d	Number of scanned dimensions during performing DCO

Remark. The scalability of exact solutions to vector similarity search is severely limited by the curse of dimensionality [19]. This has led to a rich line of research focused on *approximate algorithms* that trade exactness for efficiency, with the primary goal of *maximizing the recall*. The recall of the query answer $\mathcal{S}$ against the ground truth $\mathcal{S}^*$ is defined as:

$$\textbf{recall} = \frac{|\mathcal{S} \cap \mathcal{S}^*|}{k} \tag{1}$$

B. Definition of Distance Comparison Operation

Overview of Vector Distance Operations. Prior research [4], [20], [21] in vector similarity search relies heavily on two types of vector distance operations: *distance computation operations* and *distance comparison operations*.

- **Distance Computation Operation**: This operation calculates the *exact distance* between two vectors. It is computationally intensive but can often be accelerated by hardware like SIMD instructions and GPUs.
- **Distance Comparison Operation (DCO)**: This operation determines whether the distance between two vectors exceeds a given threshold. Its efficiency enhancement stems from a key insight: *computing exact distance is often unnecessary*, since the comparison result can be derived by scanning only a subset of the dimensions. We formally define this operation below.

Definition 3 (Distance Comparison Operation (DCO) [1]): Given two vectors o, q and a distance threshold τ, the distance comparison operation (DCO) determines the true value of the statement $dis(o, q) \leq \tau$. If true, it also returns the exact distance. Otherwise, it returns false alone, without the distance.

The distance threshold τ is usually set to the distance of candidate vectors to the query vector. As [1]–[3] identifies, the vast majority of DCOs within a vector search return false. This makes the overall cost of exact distance calculation relatively low, as it is incurred only for the small subset of DCOs that return true. The following example illustrates this operation.

Example 1: Fig. 1 shows two instances of DCO with data vectors o_1 and o_2, a query vector q, and a distance threshold $\tau = 11$. A naive solution is to scan all dimensions, compute the

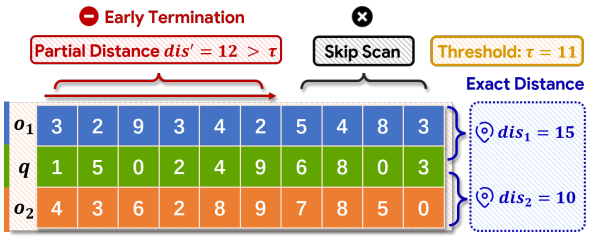


Fig. 1: Instances of Distance Comparison Operation (DCO)

Euclidean distances $dis(o_1, q) = 15$ and $dis(o_2, q) = 10$, and then compare them with τ. *However, a full-dimensional scan is not always necessary.* For o_1, after scanning just the first 6 coordinates, the partial distance reaches 12, which already exceeds τ. This eliminates the need to scan the remaining dimensions of o_1 and thereby reduces computational cost.

III. Existing DCO Methods

In this section, we first introduce a taxonomy of existing DCO methods, and categorize them as *simple scanning based*, *hypothesis testing based*, and *classification based* methods. We then review each category and finally present a comparative analysis of these DCO methods.

A. Method Taxonomy

The design of high-performance DCOs follows two paths: *optimized algorithms* and *dedicated hardware*. Hardware-based approaches usually leverage GPUs [22] for massive thread-level parallelism or SIMD instructions [23] for dimension-level parallelism. However, as recent research focuses on algorithmic optimization strategies [1]–[3], [24], these hardware-centric methods are not our primary concern.

As shown in Fig. 2, we categorize these optimization algorithms into three kinds based on their core ideas: *simple scanning based*, *hypothesis testing based*, and *classification based*. These methods are complementary to hardware accelerators like GPUs and SIMD instructions, which will be evaluated later. The subsequent subsections will delve into each of these categories in detail, and Fig. 3 provides a high-level overview of their respective ideas and workflows.

B. Simple Scanning Based Method

Simple scanning based methods solve the DCO problem by scanning dimensions of vectors o and q, through two strategies: *Full Dimension Scanning* (*FDScanning*) and *Partial Dimension Scanning* (*PDScanning*).

- FDScanning [23] scans all dimensions to compute the exact distance and then compares it to the threshold τ.
- PDScanning [25] scans dimensions incrementally while maintaining a running partial distance. It terminates the scan as soon as the intermediate result proves that $dis(o, q) > \tau$, avoiding unnecessary computation.

General Framework. Alg. 1 presents the general framework of these DCOs. It starts scanning dimensions from scratch, keeping track of the partial distance dis' so far. The main difference between FDScanning and PDScanning lies in when

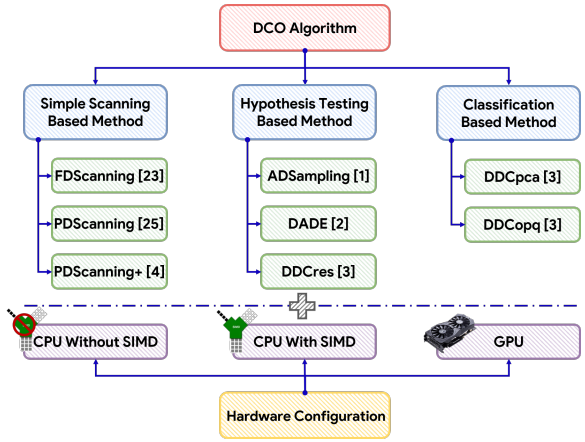


Fig. 2: Taxonomy of existing DCO methods

Algorithm 1: Simple Scanning Based Framework

Input: Two vectors o, q and a distance threshold τ
Output: Whether $dis(o, q) \leq \tau$

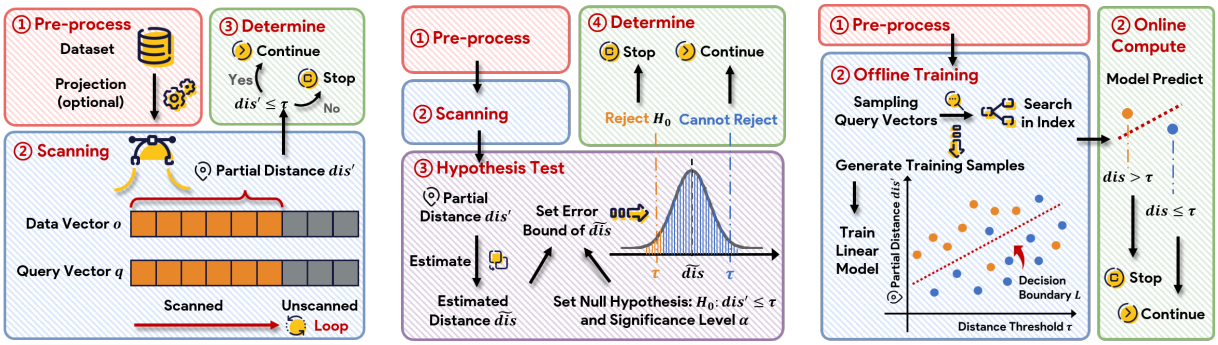
1 Initialize the number of currently scanned dimensions $d \leftarrow 0$, partial distance $dis' \leftarrow 0$;
2 **while** *the number of scanned dimensions* $d < D$ **do**
3 Update partial distance dis' and increase d;
4 **if** $dis' > \tau$ **then** // `only for` PDScanning
5 **return** *False*;
6 **return** *True and* $dis \leftarrow dis'$;

they stop: PDScanning quits early as soon as dis' exceeds the threshold τ, while FDScanning scans all dimensions.

Optimization. The efficiency of this framework can be further enhanced by two techniques: Single Instruction Multiple Data (SIMD) and Principal Component Analysis (PCA) [26].

(1) Optimization via SIMD: The partial distance computation can be parallelized at the dimension-level using SIMD. For example, both FDScanning and PDScanning can partition the dimensions of a vector into contiguous blocks that align with the SIMD register width (*e.g.*, 128-bit or wider). This enables the CPU to simultaneously perform arithmetic operations across multiple dimensions within a single instruction cycle. In this way, more than 4 dimensions can be processed together in a batch, which saves computational cost.

(2) Optimization via PCA: PCA projects the original vector data into a new coordinate system, where dimensions are ordered by their variance contributions. By rotating the original vector space (rather than performing dimensionality reduction), this transformation preserves the overall Euclidean distances between vectors while prioritizing the dominant influence of the leading dimensions. PDScanning can also leverage this property to enable early termination in lines 4-5 of Alg. 1. We denote this method as PDScanning+ [4] in our work. Despite its simplicity, it has not been considered as a candidate DCO method in prior studies [1]–[3], primarily because FDScanning and PDScanning are prevalent in vector databases. However, we treat it as a distinct baseline, since it offers unique advantages according to our evaluations.



(a) Simple scanning based method (b) Hypothesis testing based method (c) Classification based method

Fig. 3: The three categories of DCO methods: core ideas and main workflows

C. Hypothesis Testing Based Method

The anticipated efficiency gain of PDScanning over FDScanning stems from scanning fewer dimensions. This intuition motivates a class of approximate algorithms that estimate the full distance (denoted by $\widetilde{dis}$) from a partial distance (dis') to scan even fewer dimensions. Using the estimated distance within a probability inequality, these algorithms then perform a hypothesis test to accept or reject $dis(o,q) \leq \tau$. They are classified into two kinds based on the estimation strategies: (1) *residual distance estimator* and (2) *cross-term estimator*.

Strategy I. Residual Distance Estimator. This class of algorithms estimates the residual distance (*i.e.*, the distance contributed by unscanned dimensions) based on the partial distance dis' from the currently scanned dimensions. There are two general methods for such estimators.

(1) Assuming Equal Contribution per Dimension. Under this assumption, the full distance can be estimated by simply scaling up the partial distance proportionally to the ratio of total dimensions to scanned dimensions. To enforce this assumption in practice, ADSampling [1] employs a random projection matrix $P \in \mathbb{R}^{d \times D}$ to map original vectors into a new coordinate system. This work also proves an error bound on the estimated distance as follows:

$$\mathbb{P}\left\{\left|\sqrt{\frac{D}{d}}dis(Po,Pq) - dis(o,q)\right| \leq \epsilon dis(o,q)\right\} \geq 1 - 2e^{-c\cdot d\cdot \epsilon^2}$$

In this inequality, $\sqrt{\frac{D}{d}}dis(Po,Pq)$ is the estimated distance $\widetilde{dis}$, c is a constant factor, and ϵ is the error tolerance. Thus, the inequality implies that $\widetilde{dis} \leq (1+\epsilon)\cdot dis$ holds with high probability $1 - 2e^{-cd\epsilon^2}$. Moreover, this estimator ensures that $\widetilde{dis} = dis$ when all original dimensions are preserved in the projection (*i.e.*, $d = D$).

(2) Assuming Differing Contribution per Dimension. In contrast, DADE [2] is designed for scenarios where dimensions contribute differently to the distance, and treats dimensions with different weights. Accordingly, it performs a PCA over the vector dataset to obtain the loading matrix $W \in \mathbb{R}^{D\times D}$ as the projection matrix. This ensures that the initial dimensions capture the largest possible contribution to the distance. Consequently, the result of DCOs can be quickly decided after scanning only a few leading dimensions.

Similarly, Deng *et al.* [2] also propose an error bound on the estimated distance as follows:

$$\mathbb{P}\left\{\sqrt{\frac{\sum_{k=1}^{D}\lambda_k}{\sum_{k=1}^{d}\lambda_k}}dis(W_d^T o, W_d^T q) > (1+\epsilon_d)dis(o,q)\right\} < \alpha \quad (2)$$

where λ_k is the k-th largest eigenvalue of the vector dataset's covariance matrix, α is the significance level, and ϵ_d is the error tolerance after scanning d dimensions. Both α and ϵ_d are user-defined parameters whose values are set empirically.

Strategy II. Cross-Term Estimator. This strategy employs an algebraic decomposition of the squared Euclidean distance into a sum of square-terms and cross-terms:

$$dis(o,q) = \|o\|^2 + \|q\|^2 - 2\langle o, q\rangle \quad (3)$$
$$= \|o\|^2 + \|q\|^2 - 2\langle od, qd\rangle - 2\langle or, qr\rangle \quad (4)$$

where $\|\cdot\|$ is the L_2-norm, and $\langle\cdot,\cdot\rangle$ is the inner product. The sub-vectors od, qd and or, qr contain the scanned and unscanned dimensions, respectively. In Eq. (4), both $\|o\|$ and $\|q\|$ can be pre-processed, and $\langle od, qd\rangle$ can be computed directly from the scanned dimensions. Their collective partial result is denoted by dis'. This method thus focuses on estimating the remaining cross-term, $\langle or, qr\rangle$.

Yang *et al.* [3] propose a new method called DDCres to bound this term. DDCres [3] first centers the data vectors to have zero mean. It then assumes that both query and data vectors follow a Gaussian distribution, where σ_i^2 denotes the variance of the i-th dimension. Accordingly, the expectation and variance of the remaining cross-term are derived as:

$$\mathbb{E}[\langle o_r, q_r\rangle] = \sum_{i=d+1}^{D}\mathbb{E}[o_i\cdot q_i] = \sum_{i=d+1}^{D}\big(\mathbb{E}[o_i]\cdot\mathbb{E}[q_i]\big) = 0 \quad (5)$$

$$\mathrm{Var}[\langle o_r, q_r\rangle] = \sum_{i=d+1}^{D}(q_i\cdot\sigma_i)^2 \quad (6)$$

Consequently, the exact distance dis lies within a confidence interval $dis' \pm m\cdot 2\sqrt{\mathrm{Var}[\langle o_r, q_r\rangle]}$ with high probability, where m is a deviation multiplier. To test if $dis > \tau$, the lower bound of this interval is used as the estimated distance $\widetilde{dis}$, *i.e.*,

$$\widetilde{dis} = dis' - m\cdot 2\sqrt{\mathrm{Var}[\langle o_r, q_r\rangle]} \quad (7)$$

To tighten this lower bound, DDCres leverages PCA to minimize Eq. (6), since PCA reorders data dimensions in descending order of σ_i^2.

Algorithm 2: Hypothesis Testing Based Framework

Input: Two vectors o, q and a distance threshold τ
Output: Whether $dis(o,q) \leq \tau$

1 Set the null hypothesis $H_0 : dis \leq \tau$ and its alternative hypothesis $H_1 : dis > \tau$;
2 **while** *the number of scanned dimensions* $d < D$ **do**
3 Update partial distance dis' and increase d;
4 Set significance level α by their parameter settings;
  // $\epsilon = \epsilon_d$ for DADE, $\epsilon = 0$ for DDCres
5 Compute a relaxed distance threshold $(1+\epsilon)\cdot\tau$;
6 Compute the estimated distance $\widetilde{dis}$ based on dis';
7 **if** $\widetilde{dis} > (1+\epsilon)\cdot\tau$ **then**
8 Reject H_0 with confidence and **return** *False*;
9 **return** *True and* $dis \leftarrow dis'$;

General Framework. Alg. 2 presents the main workflow of hypothesis testing based methods. Unlike simple scanning based methods, these methods incorporate an extra step (lines 4-6) in which hypothesis testing is used to check if the null hypothesis is valid. Correspondingly, the condition in line 7 is replaced by a verification on whether the estimated distance $\widetilde{dis}$ exceeds a statistically derived error bound. If it does, the null hypothesis is rejected and the loop terminates early.

Beyond Euclidean Distance. The hypothesis testing based methods also support inner product and cosine similarity by transforming them into Euclidean distance. This requires the vector dataset to be normalized first. Then, the inner product $\langle o, q\rangle$ is derived from the Euclidean distance $dis(o,q)$ as:

$$\langle o, q\rangle = 1 - 0.5 \cdot (dis(o,q))^2 \tag{8}$$

Since cosine similarity is equivalent to the inner product for normalized vectors, this extension applies to both metrics.

D. Classification Based Method

Main Idea. Other research [3] formulates the DCO problem as a binary classification task. Given the partial distance dis' and threshold τ, the goal of this task is to predict whether the full distance satisfies $dis \leq \tau$. These methods typically assume the query workload is known a priori (*e.g.*, often modeled using the vector dataset's distribution). Under this assumption, they utilize a fixed vector index and sampled queries (including both an integer k and a query vector q) to generate sufficient training samples. Subsequently, a distinct linear model $M_{k,d}$ is trained for each combination of the integer k and the number d of scanned dimensions.

Representative Algorithm. Alg. 3 illustrates a representative algorithm DDCpca to this kind. In the offline phase, it runs similarity searches with different k values on a fixed vector index to generate training samples. These samples are used to train a set of linear models $M_{k,d}$, each corresponding to a query parameter k after d dimensions are scanned. During the online phase, DDCpca feeds the partial distance dis' and distance threshold τ into the model $M_{k,d}$. The process terminates early if the model predicts $dis(o,q) > \tau$.

Algorithm 3: Classification Based Method DDCpca

Input: Two vectors o, q and a distance threshold τ
Output: Whether $dis(o,q) \leq \tau$

1 // **Offline Phase**
2 Sample query vectors from dataset and parameters k;
3 Generate training samples through vector similarity search for sampled queries using a specific index;
4 Train linear models $M_{k,d}$, denoting the model for query parameter k after d dimensions are scanned.;
5 // **Online Phase**
6 **while** *the number of scanned dimensions* $d < D$ **do**
7 Update partial distance dis' and increase d;
8 **if** $M_{k,d}$ *predicts* $dis(o,q) > \tau$ **then return** *False* ;
9 **return** *True and* $dis \leftarrow dis'$;

Variant. Yang *et al.* [3] propose another classification based method, named DDCopq. Unlike DDCpca, DDCopq trains a single linear model M_k for each query parameter k, using approximate distances derived from Product Quantization (PQ). During a DCO, this model first predicts if $dis(o,q) > \tau$. If the result is negative, it performs a distance computation operation by scanning all dimensions to verify the result.

E. Summary

Table III compares DCO methods on the following aspects:

(1) Exactness: Simple scanning based algorithms are exact, while the others trade slight accuracy for efficiency. Besides, only DADE provides an unbiased distance estimator.

(2) Assumption: DCO methods vary in their assumptions. Most DCO methods are limited to Euclidean distance (or transformable metrics), whereas PDScanning and PDScanning+ also support monotonic distances. DADE, DDCres, DDCpca, and DDCopq assume queries and data follow the same distribution. DDCres further assumes this distribution is Gaussian. Classification based methods require prior knowledge of the query parameter k and are coupled to a specific index.

(3) Time Complexity: Only FDScanning, PDScanning, and ADSampling provide explicit time complexity analyses. However, ADSampling underestimates the $O(D^2)$ time cost of projecting a query vector into a new coordinate system. We categorize this per-query operation as *online pre-processing*, a cost also incurred by PDScanning+, hypothesis testing based, and classification based methods. Our analysis introduces d to denote the number of dimensions scanned and p_i as the probability of a negative prediction. We also include the time cost of *offline pre-processing*, which includes PCA computation and model training. Our results show that **online pre-processing can become the dominant bottleneck at sufficiently high dimensions**, a finding that will be validated empirically later.

IV. Benchmark Setup

This section introduces the detailed benchmark setup. We make our benchmark suite publicly available on GitHub [10].

A. Overview

Our benchmark covers the following critical scenarios:

TABLE III: Comparisons of existing methods for Distance Comparison Operation (DCO)

Category	Method	Exactness	Assumption[1]	Time Complexity Breakdown[2]		
				Offline Pre-Processing	Online Pre-Processing	Online Computation
Simple Scanning	FDScanning [23]	✓	None	N/A	N/A	$O(D)$
	PDScanning [25]	✓	Monotonic Distance	N/A	N/A	$O(\min(D, D\frac{\tau^2}{dis^2}))$
	PDScanning+ [4]	✓	Monotonic Distance	$O(ND^2)$	$O(D^2)$	$O(d)$
Hypothesis Testing	ADSampling [1]	×	Euclidean	$O(ND^2)$	$O(D^2)$	$O(\min(D, \frac{\tau^2}{(dis-\tau)^2}\log\frac{D}{\delta}))$
	DADE [2]	×	Euclidean, Query	$O(ND^2)$	$O(D^2)$	$O(d)$
	DDCres [3]	×	Euclidean, Data, Query	$O(ND^2)$	$O(D^2)$	$O(d)$
Classification	DDCpca [3]	×	Euclidean, Index, Query	$O(ND^2)$	$O(D^2)$	$O(\sum_{j=1}^{D}\prod_{i=1}^{j-1}(1-p_i))$
	DDCopq [3]	×	Euclidean, Index, Query	$O(ND(D+2^b))$	$O(D^2+D\cdot 2^b)$	$O(c+(1-p)D)$

[1] "Euclidean": only applicable to Euclidean distance or distances that can be transformed into it.; "Data": data vectors follow a known distribution (*e.g.*, Gaussian); "Query": query vectors follow the data distribution; "Index": requires building a vector index for training.
[2] δ: failure probability for $(1+\epsilon)$-approximation; b, c: product quantization parameters for subvector encoding length and codebook count.

- **Query Processing**: Evaluating how DCOs impact vector similarity search performance across diverse datasets.
- **Out-of-Distribution (OOD) Queries**: Generalization when query distribution differs from the data distribution.
- **Distance Metrics**: Evaluating DCO performance on Euclidean distance, inner product, and cosine similarity.
- **Index Construction**: The efficacy of DCOs in building mainstream vector indexes: HNSW [27] and IVF [28].
- **Dynamic Updates**: The robustness and effectiveness of DCOs when handling dynamic data.
- **Diversified Hardware Condition**: The performance of DCOs under varied hardware configurations.

TABLE IV: Dataset statistics

Dataset	Dim.	Category	Raw Data	Cardinality	Size (GB)
Deep	96	Low-D	Image	100,000,000	36
GloVe	100	Low-D	Text	400,000	0.15
SIFT	128	High-D	Image	1,000,000	0.5
Text2Image	200	High-D	Multimodal	10,000,000	7.5
Laion	512	High-D	Multimodal	1,000,448	2.0
Wikipedia	768	High-D	Text	1,000,000	2.9
GIST	960	High-D	Image	1,000,000	3.6
OpenAI	1536	Ultra-High-D	Text	2,321,096	14
Trevi	4096	Ultra-High-D	Image	99,900	1.6
XUltra	12288	Ultra-High-D	Text	100,000	4.6

B. Dataset and Query Workload

Our benchmark employs 10 public datasets [29]–[32] for evaluating vector similarity search with varying dimensions, sizes, and raw data types (see Table IV). Most datasets contain in-distribution queries, which either share a distribution with the dataset or are sampled from it. In contrast, the multimodal datasets (Laion and Text2Image) exhibit an inherent distribution shift, as their data and query vectors are embedded from images and text, respectively, making their queries OOD.

To simulate the eXtremely Ultra-high dimensionality of embeddings in modern LLMs (*e.g.*, OpenAI's davinci-001 model with 12,288 dimensions [33]), we also construct a synthetic dataset named XUltra. It is generated by concatenating token-level embeddings from the real-world dataset MSMARCO [32] until the target dimensionality of 12,288 is reached. In this dataset, each resulting vector represents the aggregation of tokens in a complete text phrase. The ground truth is obtained by exhaustive linear scan of the full dataset.

C. Compared DCO Algorithms and Their Implementations

We implement 8 DCO methods from prior research [1]–[4]: FDScanning, PDScanning, PDScanning+, ADSampling, DADE, DDCres, DDCpca, and DDCopq. The first three, serve as *baselines*, widely adopted in industrial vector databases. The other five are the *state-of-the-art* algorithms ("SOTA" as short). For a fair comparison, we implement all algorithms from scratch within a unified framework:

- **Index Selection**: Each method is integrated into two indexes using identical data layouts: HNSW [27] and IVF [28], with HNSW running on CPUs and IVF on GPUs.
- **Target Hardware**: We implement SIMD-optimized and GPU-accelerated versions for all algorithms. On CPUs, we evaluate these algorithms with and without SIMD optimizations, which accelerate distance computations via intra-vector parallelism. On GPUs, we pre-loaded IVF into device memory and adopt vector-level parallelism, launching a CUDA kernel for each IVF partition with each thread assigned to process one candidate vector.
- **Parameter Settings**: DCO methods are configured with parameters recommended in their original papers.
- **Batch Query**: Batch queries are processed sequentially.

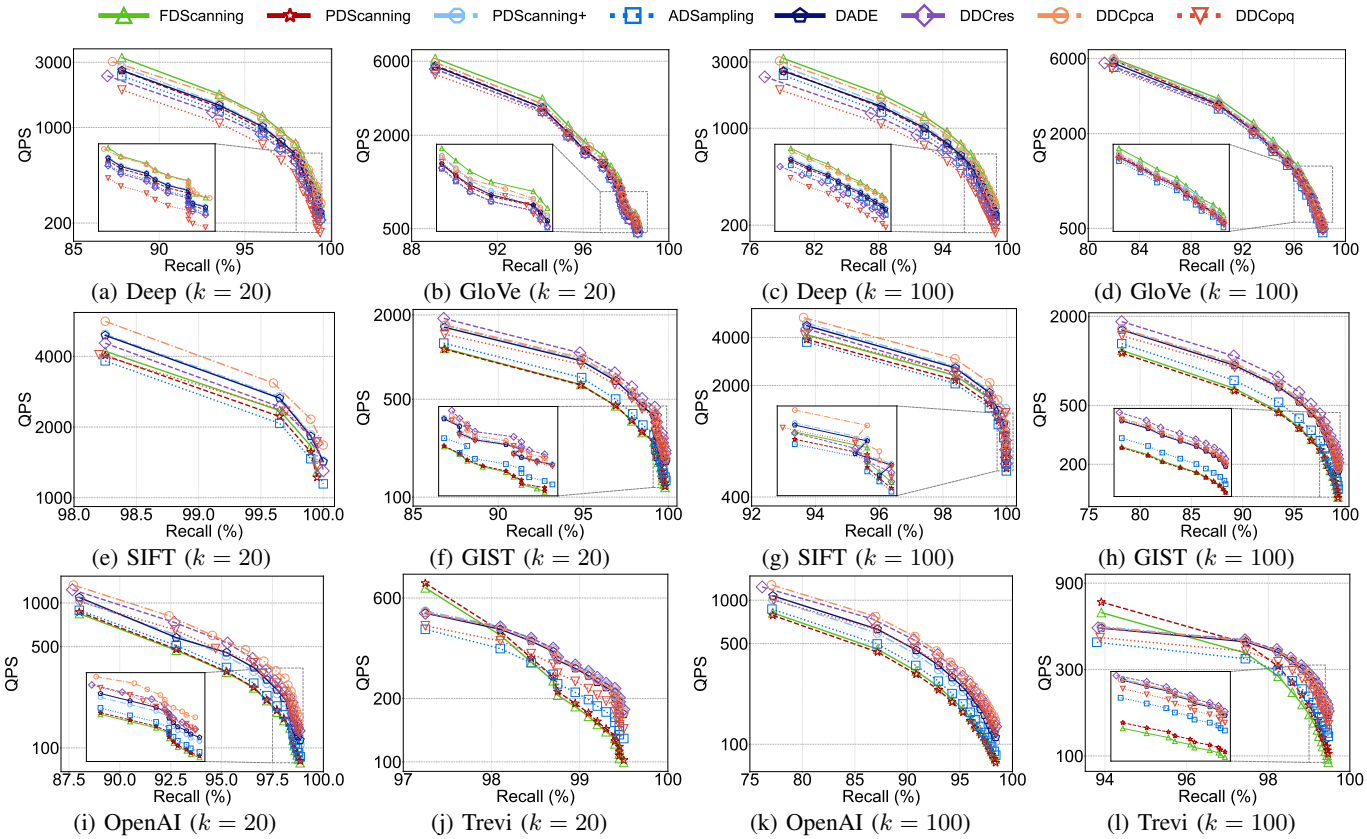
We implement all methods in both Python and C++, following their open-source implementations. Online query processing is written in C++, while offline pre-processing uses both C++ and Python, where Python is mainly employed for model training and PCA computation. By default, we enable SIMD via the SSE instruction set and keep multi-threading disabled. Please refer to [34] for more implementation details.

D. Parameter Setting

The benchmark setup involves three types of parameters:

Index Parameter. We configure HNSW with maximum node connections $M = 16$ and construction candidate list size $efConstruction = 500$, and IVF with 4096 partitions, following recommendations from Faiss [23].

Search Parameter. For HNSW, we vary the search candidate list size $efSearch$ from 100 to 1500 in steps of 100. For IVF, we vary the probed partition count $nprobe$ from 20 to 400 in steps of 20. By default, $efSearch = 200$ and $nprobe = 80$.



(a) Deep ($k = 20$) (b) GloVe ($k = 20$) (c) Deep ($k = 100$) (d) GloVe ($k = 100$)

(e) SIFT ($k = 20$) (f) GIST ($k = 20$) (g) SIFT ($k = 100$) (h) GIST ($k = 100$)

(i) OpenAI ($k = 20$) (j) Trevi ($k = 20$) (k) OpenAI ($k = 100$) (l) Trevi ($k = 100$)

Fig. 4: Query performance when DCOs are applied to vector similarity search

Query Parameter. We set the integer k to two common values, 20 and 100, as used in prior work [20], [35], [36].

E. Evaluation Metric

We assess DCOs from the following four perspectives:

- **Effectiveness**: Evaluated via **recall** defined in Def. 1.
- **Query Throughput**: Measured by queries per second (**QPS**). As each query typically invokes the DCO multiple times, QPS represents the overall efficiency of DCOs.
- **Pruning Capability**: Quantified by the **dimension pruning ratio**, *i.e.*, the fraction of dimensions saved.
- **Construction & Insertion Overhead**: Measured by **index construction time** and **data insertion time**.

Following prior work [1]–[3], we plot QPS–recall curves by varying the index search parameters $efSearch$ and $nprobe$ for HNSW and IVF, respectively. Increasing these parameters raises recall while concurrently decreasing QPS.

F. Experimental Environment

Experiments are conducted on a server with an AMD Ryzen 9 5950X CPU, 128GB RAM, and an NVIDIA GeForce RTX 3090 GPU. The server ran Ubuntu 24.04.2 with CUDA 12.4.

V. Experimental Evaluation

This section presents our experimental results and analysis. Due to page limits, we show only selected results here. Please refer to the full paper [34] for complete results (*e.g.*, the parameter study on the number of scan dimensions per round and the evaluation using the AVX2 SIMD instruction set).

A. Query Processing

This experiment evaluates the performance improvement from applying DCOs to vector similarity search for $k = 20$ and $k = 100$. We use seven datasets with varying dimensions (96 to 12,288) and cardinalities (99,900 to 100 million) to ensure a comprehensive assessment. Fig. 4 and 5 present time–recall curves to demonstrate the effectiveness of DCO methods.

Overall Query Performance. The results show that in only 4 out of the 14 subfigures, the hypothesis testing based and classification based methods *consistently outperform* the baseline FDScanning. Specifically, compared to FDScanning, the hypothesis testing based methods improve QPS by up to 1.4–1.9×, while DDCpca and DDCopq achieve improvements of up to 1.6–2.1×. These results demonstrate the superior enhancement in time efficiency when applying DCOs to vector similarity search, while their recall remains largely stable with at most 2% change.

We also observe cases where naive baselines (*e.g.*, FDScanning) perform better than the others. For instance, Fig. 4a and 4c show that FDScanning usually achieves the highest QPS on the Deep dataset. Similarly, Fig. 4j and 4l show that PDScanning obtains the optimal efficiency for recall until 97% and 94% respectively on the Trevi dataset.

The SOTA methods become ineffective on the Deep and Trevi datasets for different reasons. On the low-dimensional dataset Deep, the time saved by skipping 6%–36% of dimensions is marginal (except for DDCopq), since the accumulated

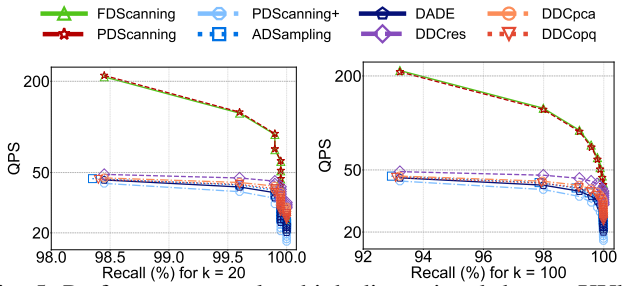
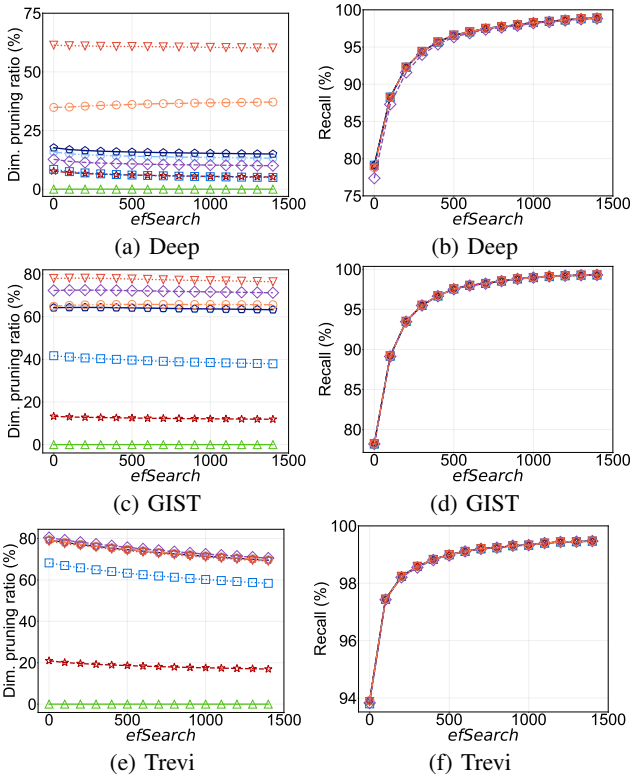


Fig. 5: Performance on ultra-high-dimensional dataset XUltra



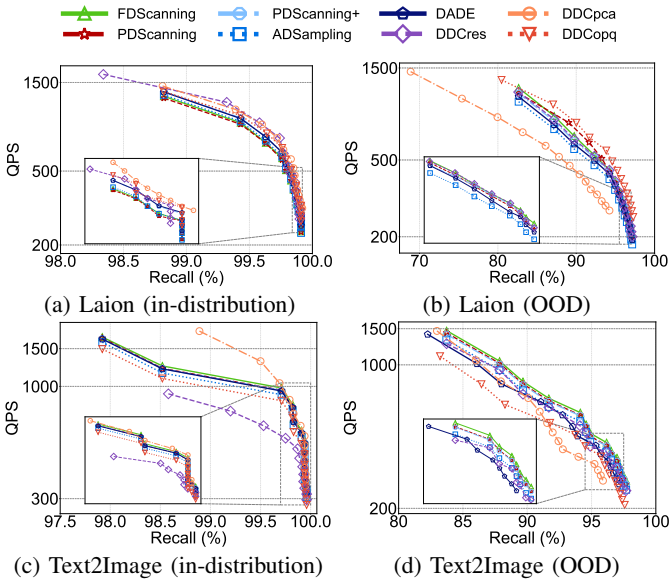
(a) Deep (b) Deep (c) GIST (d) GIST (e) Trevi (f) Trevi

Fig. 6: Dimension pruning via DCO and its impact on recall

overhead of these methods including distance estimation, threshold checking, and conditional branching outweighs the benefits of dimension pruning. On the ultra-high-dimensional dataset Trevi, the $O(D^2)$ online pre-processing time per query becomes the efficiency bottleneck. This cost is not amortized effectively before the recall reaches 94%, where DCOs are only invoked for $O(\log N)$ candidate vectors on HNSW [27].

This phenomenon becomes increasingly pronounced at higher dimensionalities. Fig. 5 shows the result on the XUltra dataset which has the highest dimensionality. Here, FDScanning and PDScanning achieve higher QPS than all SOTA methods for both k values. The QPS of these SOTA methods decreases by up to 77%–79%$\times$ compared to FDScanning.

Pruning Capability of DCO. Fig. 6 illustrates the dimension pruning ratios of DCOs and their recall across three datasets of varying dimensionality. The results demonstrate that the pruning capabilities of DCOs are dimension-dependent: DDCopq achieves the highest pruning ratio on the Deep and GIST datasets, while DDCres performs best on the other dataset



(a) Laion (in-distribution) (b) Laion (OOD) (c) Text2Image (in-distribution) (d) Text2Image (OOD)

Fig. 7: Results on in-distribution and OOD queries

with higher dimensions. On Deep, the pruning capability of ADSampling is nearly equivalent to that of PDScanning, which is consistent with its suboptimal performance in Fig. 4. Moreover, all methods maintain a comparable level of recall. This consistent result indicates that the minor approximation errors of the DCO solutions do not compromise recall.

Takeaway #1: DCO is no silver bullet. While adept at skipping unnecessary dimensions with minimal recall loss, the SOTA DCO methods do not universally boost efficiency. Our evaluation shows that they can even be slower than the native baseline FDScanning, especially on low- and ultra-high-dimensional datasets.

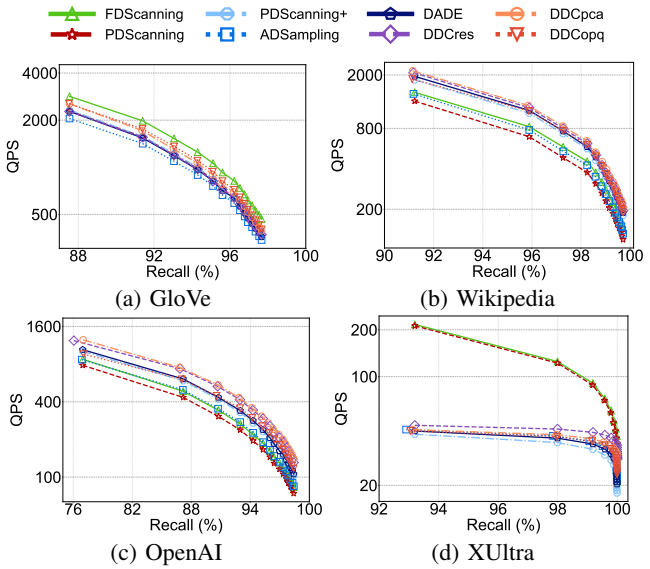
B. Out-of-Distribution (OOD) Queries

To evaluate the robustness of DCOs for OOD queries, we conduct experiments on two text-to-image datasets: Laion and Text2Image. We also include in-distribution queries by sampling query vectors directly from image embeddings.

In Fig. 7, FDScanning and PDScanning, consistently achieve higher QPS than most SOTA methods on OOD queries. For example, DDCpca achieves up to 1.5$\times$ higher QPS than FDScanning on in-distribution queries, its performance degrades significantly on OOD queries, becoming up to 1.6$\times$ slower than it. This is primarily due to two factors: (1) the reduced prediction accuracy of DDCpca necessitates more candidate vectors to maintain recall, and (2) the dimension pruning ratio of most DCO methods drops by over 60% on OOD queries. By contrast, DDCopq achieves the highest QPS on the Laion dataset. This advantage stems from its use of quantized distances, which maintain high prediction accuracy (over 95%) while also being accelerated by SIMD instructions.

Furthermore, even for in-distribution queries, FDScanning and PDScanning outperform PDScanning+, ADSampling, DADE, DDCres, and DDCopq in QPS. This result confirms the sensitivity of most DCO methods to data dimensionality.

Takeaway #2: SOTA methods often fail to maintain efficiency gains on Out-of-Distribution (OOD) queries. In particular, no SOTA algorithms consistently achieve higher QPS than FDScanning on multimodal datasets.



(a) GloVe (b) Wikipedia (c) OpenAI (d) XUltra

Fig. 8: Performance on distance metric of Inner Product (IP)

C. Beyond Euclidean Distance

This experiment evaluates extensions of DCOs to non-Euclidean distance (inner product), as all DCOs support it. As shown in Fig. 8, we conduct comprehensive evaluations using four datasets with distinct dimensionality.

The similar trends observed in Fig. 4 and 8 are expected, as most DCO methods convert IP into a Euclidean distance computation. On the OpenAI dataset, DADE, DDCres, DDCpca, and DDCopq improve QPS by up to 1.3–1.7× compared to FDScanning, which demonstrates their ability to handle IP. However, there are also cases on GloVe and XUltra datasets, where FDScanning outperforms all the other methods. For instance, it achieves the highest QPS on GloVe, outperforming the SOTA methods by up to 1.2–1.4×. Meanwhile, the bottleneck caused by online preprocessing time persists: the QPS of these SOTA methods drops by up to 78%–79% compared to FDScanning across these datasets. Notably, a similar trend is observed for cosine similarity. Please refer to our full paper [34] for detailed results of cosine similarity.

Takeaway #3: The similar performance of DCOs under IP and Euclidean metrics stems from their reliance on converting the former to the latter, but this comes at the cost of generality.

D. Index Construction

While most DCOs have the potential to accelerate index construction, this capability has not been systematically evaluated in prior work. We fill this gap by evaluating their performance, excluding the classification based methods (DDCpca and DDCopq) that themselves require an index for training.

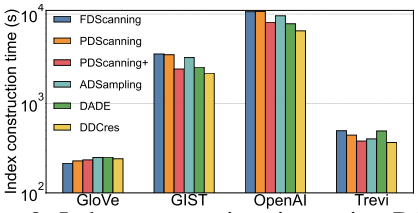


Fig. 9: Index construction time using DCO

Index Construction Time. Fig. 9 shows the index construction time with DCOs. On the low-dimensional dataset GloVe, construction time even increases with the SOTA methods. This further confirms that most DCO methods do not perform well in datasets with comparably lower dimensions. By contrast, on high-dimensional datasets (GIST and OpenAI), most DCO methods achieve acceleration, including PDScanning+, ADSampling, DADE, and DDCres. For example, DDCres reduces index construction time by up to 39% across these datasets. On the Trevi dataset, most DCO methods also achieve acceleration. The pre-processing time for PCA is lower than that required for HNSW index construction, the latter of which typically constitutes over 60% of the total runtime.

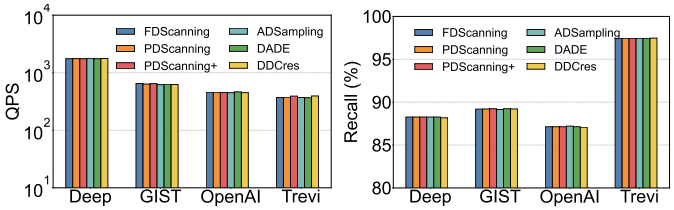


Fig. 10: Performance evaluation of different indexes

Vector Similarity Search Performance. As most DCOs are approximate, they may result in different indexes. We evaluate the constructed indexes on vector similarity search using a fixed query processing method (FDScanning) for a fair comparison. Fig. 10 shows that all indexes achieve nearly identical average recall and QPS across different datasets, where their recall gap to FDScanning is no more than 0.1%. This demonstrates that indexes built with DCOs effectively maintain end-to-end vector search performance.

Takeaway #4: DCOs can accelerate index construction without compromising search performance, but their efficacy mainly depends on data dimensionality.

E. Robustness on Dynamic Data

While DCO methods have demonstrated strong performance on certain static datasets, their behavior on dynamic data remains unknown due to limited evaluations on this scenario. We fill this gap by conducting experiments on incrementally inserting data. We focus solely on insertion because deletion mechanisms are implemented independently of the DCO.

Stability Under Sequential Data Insertion. We evaluate DCO methods during incremental insertions on the GIST dataset. This dataset is split into a 60% base set for initial HNSW construction and a 40% incremental set inserted in four batches. When inserting a new vector, HNSW must search for neighbors at each layer. During the search, we replace

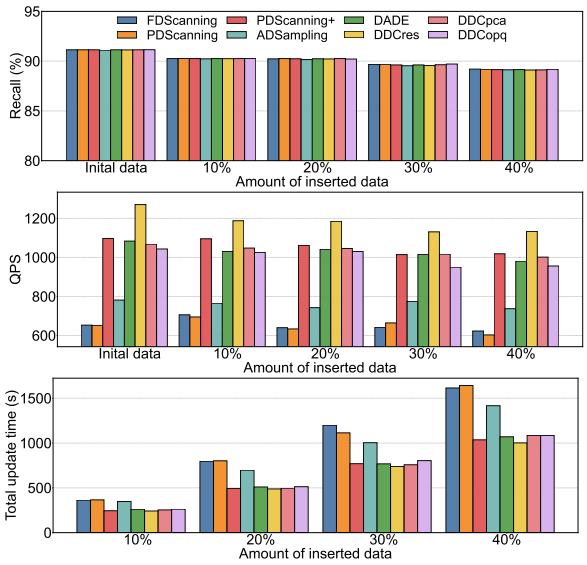


Fig. 11: QPS, recall, and total update time of DCO methods as new vectors are incrementally inserted into HNSW

FDScanning with various DCO methods to evaluate their impact on insertion performance. The resulting QPS, recall, and total update time after each batch are reported in Fig. 11.

As the dataset grows, the recall of all methods declines due to the increased search space. Similarly, the QPS of some methods (*e.g.*, DDCres) decreases, while others (*e.g.*, ADSampling) remain relatively stable. Among the compared algorithms, DDCres exhibits the most noticeable drop in QPS, reaching up to 11%. Meanwhile, most DCO methods reduce the data insertion time. For example, DDCres reduces the total update time by up to 39%. This shows that DCOs provide the flexibility to maintain efficiency in dynamic data scenarios.

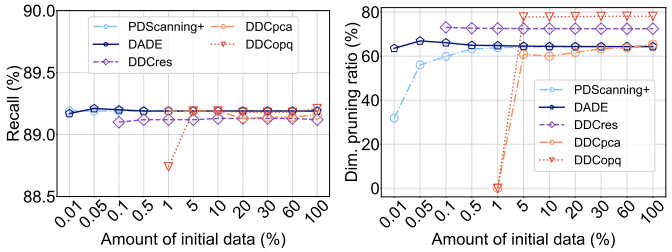


Fig. 12: Performance with limited initial data

Performance with Limited Initial Dataset. Most methods, except for FDScanning and PDScanning, rely heavily on the initial data for training prediction models or computing PCA matrices. To evaluate their sensitivity, we test their performance across a wide range of initial dataset sizes (*i.e.*, from 0.01% to 100% of the full dataset) and report the resulting recall and dimension pruning ratio in Fig. 12.

Specifically, with only 1% of the full data as the training set, PDScanning+, DADE, and DDCres achieve effective pruning performance. They demonstrate strong adaptability to real-world scenarios, where only a small fraction of data is available at the start and more is appended over time. In contrast, DDCopq and DDCpca require at least 5% of the full dataset to reach a relatively stable recall and pruning ratio. Moreover, both methods require a minimum of 10,000 vectors for training, which explains why their results are absent when the initial dataset is below 1%.

Takeaway #5: Most DCO methods maintain their performance under dynamic datasets and reduce insertion time, but classification based methods are suboptimal with limited initial training data.

F. Diversified Hardware Condition

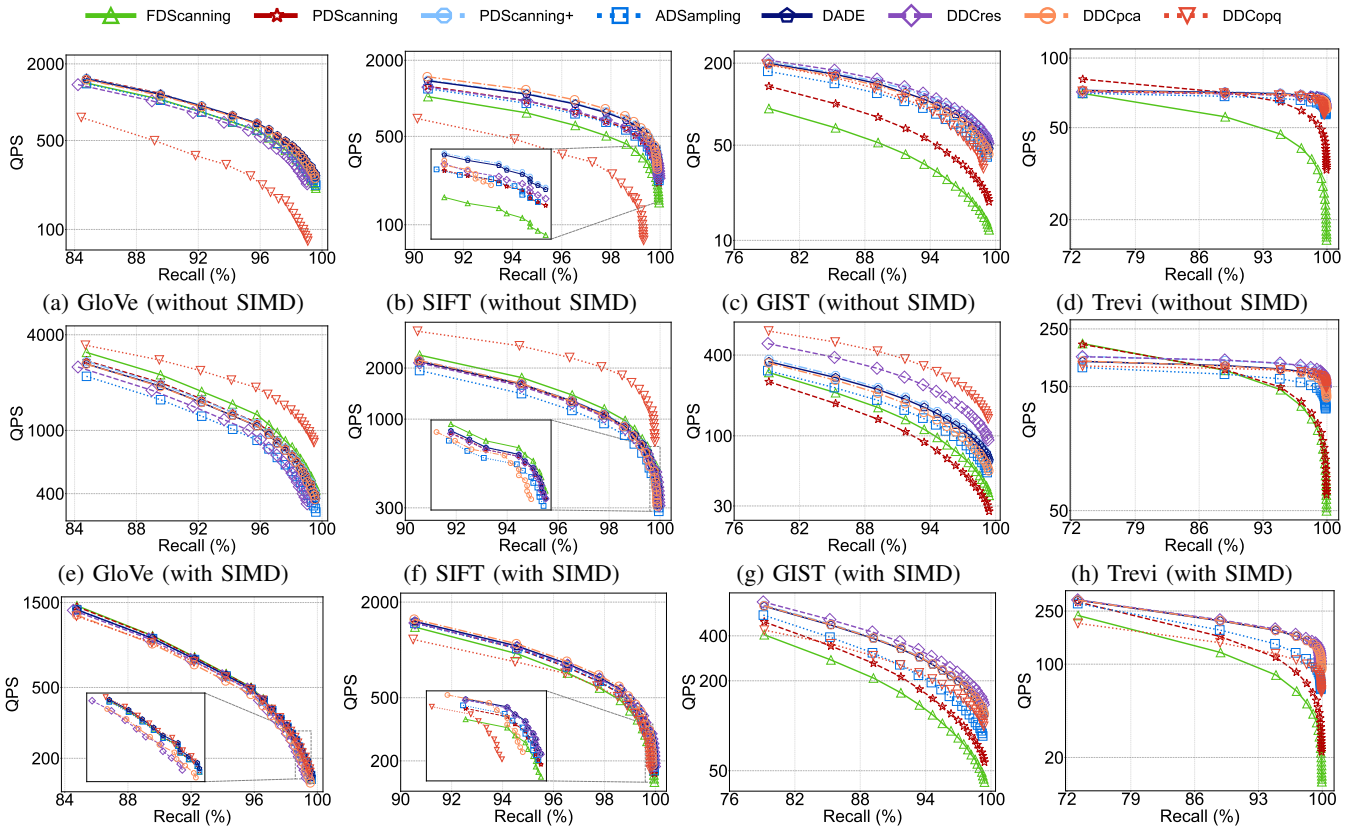
We evaluate DCO methods across different hardware platforms: GPU and CPU (with/without SIMD support). Consequently, we use the IVF index for this experiment, as HNSW often lacks native GPU support.

Performance on CPU with/without SIMD Support. As illustrated in the first two rows of Fig. 13, the usage of SIMD significantly alters the performance ranking of DCO methods in terms of QPS. For example, on the SIFT dataset, DDCopq is the least efficient method with SIMD disabled, but becomes the most efficient when SIMD is enabled. This is because SIMD enables the parallel computation of multiple quantization distances by packing them into a single instruction. The simple scanning based method PDScanning+ consistently maintains a high QPS on the Trevi dataset, where the gap to the optimal one is usually within 1% for recall over 89%.

We also observe that SIMD provides a substantial QPS boost for all methods. For example, FDScanning exhibits a QPS increase of up to 3.3× on the GIST dataset when using SIMD. Given that modern AMD and Intel processors usually support SIMD instruction sets like SSE and AVX [37], this improvement is almost a "free lunch". This justifies our benchmark to enable SIMD as the default setting. However, enabling SIMD narrows the efficiency advantage of DCO methods over FDScanning, reducing the speedup from 2.0–4.4× to 1.2–3.8× on the IVF index. This is because SIMD accelerates the vector distance computation for both FDScanning and other DCO methods, whereas the extra computational overhead of the latter erodes their relative gain. On HNSW, the speedup reduces from 1.9–3.1× to 1.1–2.1× (see full paper [34]).

Performance on GPU. The third row of Fig. 13 plots the QPS-recall curves of DCO methods running on GPU. Most methods exhibit similar performance on the GloVe dataset, while DDCres and DDCpca underperform the others. For other datasets at recall above 73%, PDScanning+, DADE, DDCres, and DDCpca achieve similar QPS, often outperforming the others. PDScanning+ performs better than ADSampling. The results of PDScanning+ demonstrate that simple scanning based methods remain competitive with GPU acceleration. Overall, ADSampling, DADE, DDCres, DDCpca, and DDCopq improve QPS with the same recall over FDScanning by up to 4.9, 7.0, 7.6, 6.9, and 4.7× respectively, across these datasets.

On the Trevi dataset, most methods achieve a higher QPS than FDScanning, a pattern that differs from the results in Fig. 4. This performance inversion is driven by the IVF index's larger number of DCO operations, which dilutes the $O(D^2)$ pre-processing cost per query. As a result, the time spent on



(a) GloVe (without SIMD) (b) SIFT (without SIMD) (c) GIST (without SIMD) (d) Trevi (without SIMD)

(e) GloVe (with SIMD) (f) SIFT (with SIMD) (g) GIST (with SIMD) (h) Trevi (with SIMD)

(i) GloVe (with GPU) (j) SIFT (with GPU) (k) GIST (with GPU) (l) Trevi (with GPU)

Fig. 13: Comparisons of DCO methods on the IVF index across different hardware configurations

online pre-processing query vectors is dwarfed by the total time of DCOs and thus does not create a bottleneck.

Performance Comparisons: CPU vs GPU. Compared to CPU without SIMD, DCO methods can improve their QPS by up to 3.0–4.3× on GPU. This improvement benefits from vector-level parallelization, which allows concurrent DCOs to be performed for a batch of query vectors. However, this advantage drops to roughly 1.1–2.2×, when compared to CPU with SIMD enabled. Based on these comparison results, we have two major observations: (1) GPU is preferable for evaluating the performance of DCOs on IVF for higher dimensional datasets, as it delivers the best performance, and (2) HNSW is better suited to CPU with SIMD, since it does not natively support GPU execution.

Takeaway #6: Hardware configuration is a critical yet overlooked factor in DCO evaluation. Hardware configuration can drastically alter performance rankings. For instance, enabling SIMD on CPU elevates DDCopq from the least to the most efficient method on the SIFT dataset.

G. Summary: Recommendations, Merits, and Limitations

Overall Recommendation. Our experimental results lead to a clear set of recommendations for selecting the optimal DCO method in different scenarios, as shown in Fig. 14. These recommendations are mainly determined by each method's average QPS ranking for achieving a recall rate over 95%.

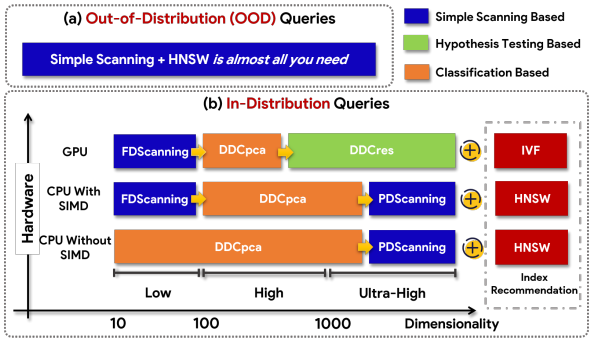


Fig. 14: Recommendations of DCO methods

The optimal DCO algorithm selection depends on three primary factors: *data dimensionality*, *hardware configurations*, and *query distribution*. For example, on CPUs with SIMD support, the best choice for in-distribution queries shifts from FDScanning to DDCpca, and then to PDScanning, as data dimensionality increases. Overall, simple scanning based methods, FDScanning and PDScanning, often outperform the others for OOD queries, and also perform well on low- and ultra-high-dimensional data for in-distribution queries. For in-distribution high-dimensional data, DDCpca is often optimal with the HNSW index on CPUs, whereas DDCres is the most efficient with the IVF index on GPUs.

Based on the above recommendations, we next analyze the main merits and limitations of existing DCO methods to guide their practical usage and future research.

Merits. Existing DCO methods offer several key advantages:

- **Enhanced Query Throughput.** These methods can substantially improve the query throughput of vector similarity search. For instance, on a CPU with SIMD enabled, PDScanning+, ADSampling, DADE, DDCres, DDCpca, and DDCopq improved QPS by up to 1.9, 1.4, 1.8, 1.9, 2.1, and 1.6× on HNSW, respectively.
- **Native Recall Preservation.** Most DCO methods preserve the native recall of underlying vector indexes across various scenarios, despite their approximate nature. For instance, hypothesis testing based methods exhibit a recall variation usually below 2% in our experimental study.
- **Accelerate Index Building & Data Insertion.** Several methods enable faster index building and data insertion.
- **Extensibility to Other Distance Metrics.** Although primarily designed for Euclidean distance, these algorithms can be adapted to other prevalent metrics while maintaining recall, such as inner product and cosine similarity.

Limitations. We also identify the following key limitations:

- **No Silver Bullet for Universal Efficiency.** The efficiency advantages of these methods are not universal but highly dependent on data dimensionality. On low- and ultra-high-dimensional datasets, most SOTA methods are slower than simple scanning based baselines, with a QPS reduction of up to 8%–42% and 77%–79% respectively.
- **Vulnerability to Out-of-Distribution (OOD) Queries.** Their performance is not robust to distribution shifts of query vectors. Experiments on multimodal datasets (Laion and Text2Image) show that simple scanning based methods (*e.g.*, FDScanning) can be more efficient than several SOTA methods, including ADSampling, DADE, DDCres, DDCpca, and DDCopq.
- **Dependence on Hardware Environment.** The efficiency gains of DCO methods are contingent on hardware. When SIMD enabled, the QPS of all methods improves, but their advantages over FDScanning narrow significantly, dropping from 2.0–4.4× to 1.2–3.8× on IVF, and from 1.9–3.1× to 1.1–2.1× on HNSW (see our full paper [34]).
- **Restrictive Assumptions Limit General Applicability.** Most DCO methods rely on specific assumptions about data distributions, query workloads, or distance metrics. For example, despite being a top performer in several scenarios, DDCpca requires prior knowledge of the query workload and parameter k, sufficient training data, and is primarily designed for Euclidean and IP distances.

VI. Related Work

We review the related work from the following two aspects: *vector similarity search* and *vector distance operation*.

Vector Similarity Search. Vector similarity search is broadly categorized into *exact* and *approximate* solutions. While exact solutions guarantee correctness, they often suffer from high query latency on large-scale datasets due to the curse of dimensionality [19]. Consequently, recent work has focused primarily on approximate algorithms [38]–[40]. A common optimization technique in these algorithms is the usage of specialized indexes, which generally fall into three categories: *graph-based indexes* (*e.g.*, HNSW [27], HVS [41], and SHG [42]), *partition-based indexes* (*e.g.*, IVF [28] and Ada-IVF [43]), and *hash-based indexes* (*e.g.*, PM-LSH [44]).

In practice, graph and partition-based indexes are the most widely adopted in production vector databases [7], [8] and search engines [9], [23]. Typical examples include HNSW [27] and IVF [28], respectively. Recent studies also explore hybrid models, such as APG-LSH [45], which integrates both graph and LSH to leverage their complementary strengths. While existing experimental studies and surveys [20], [21], [38], [46], [47] have primarily focused on index structures, our work shifts the focus to its foundation operation (see below).

Vector Distance Operation. Query processing in these indexing-based solutions involves numerous *vector distance operations*, such as vector distance computation, comparison [1]–[5], quantization [48], [49], and fusion [50], [51].

Among these operations, DCO is our primary concern. Although straightforward, the DCO methods, FDScanning and PDScanning, are commonly employed in vector databases and search engines [7], [23], [25], [27], [52], [53]. Moreover, PDScanning has also been applied in approximate nearest neighbor search in near-data processing architecture [5]. PDScanning+ [4] applies PCA in PDScanning to prioritize dimensions with greater impact on distances. While Gao *et al.* [1] first introduced hypothesis testing to support DCOs, Deng *et al.* [2] and Yang *et al.* [3] later built upon this technique to propose more effective solutions. Furthermore, Yang *et al.* [3] formulated DCO as a classification problem, and developed two classification based methods: DDCpca and DDCopq.

VII. Conclusion and Future Direction

Distance comparison operation (DCO) has emerged as a promising method for accelerating vector similarity search. While prior work has introduced several effective DCO algorithms, their evaluations have often been conducted in restricted settings, failing to fully reveal their metrics and expose their limitations under broader yet realistic conditions. To bridge this gap, we have conducted a comprehensive benchmark on eight state-of-the-art (SOTA) methods across ten diverse datasets. Our evaluation demonstrates that *no current method represents a silver bullet*: for out-of-distribution (OOD) queries, or on vector datasets with relatively low or ultra-high dimensionality, SOTA methods can even increase query latency compared to naive baselines.

Motivated by these observations, we identify the following research directions for future work:

(1) Developing specialized optimization methods tailored to relatively low- or ultra-high-dimensional datasets.

(2) Improving robustness against OOD queries, which are prevalent in real-world applications like multimodal retrieval.

(3) Incorporating hardware configuration (*e.g.*, GPU and SIMD) as a primary design consideration.

(4) Extending the scope of DCOs beyond Euclidean metrics and other limiting assumptions to enhance generality.

AI-Generated Content Acknowledgement

During the preparation of this work, we used DeepSeek-R1 in order to assist with proofreading and correction of grammatical errors and typos throughout the entire manuscript. This AI tool was applied to main sections of the article at a superficial, non-substantive level to improve readability. The authors reviewed and edited all AI-suggested changes and take full responsibility for the content of the manuscript.

References

[1] J. Gao and C. Long, "High-dimensional approximate nearest neighbor search: with reliable and efficient distance comparison operations," *Proc. ACM Manag. Data*, vol. 1, no. 2, pp. 137:1–137:27, 2023.

[2] L. Deng, P. Chen, X. Zeng, T. Wang, Y. Zhao, and K. Zheng, "Efficient data-aware distance comparison operations for high-dimensional approximate nearest neighbor search," *PVLDB*, vol. 18, no. 3, pp. 812–821, 2024.

[3] M. Yang, W. Li, J. Jin, X. Zhong, X. Wang, Z. Shen, W. Jia, and W. Wang, "Effective and general distance computation for approximate nearest neighbor search," in *ICDE*, 2025, pp. 1098–1110.

[4] Z. Wang, H. Xiong, Q. Wang, Z. He, P. Wang, T. Palpanas, and W. Wang, "Dimensionality-reduction techniques for approximate nearest neighbor search: A survey and evaluation," *IEEE Data Eng. Bull.*, vol. 48, no. 3, pp. 63–80, 2024.

[5] Y. Li, Y. Jin, B. Tian, H. Zhang, and M. Gao, "ANSMET: approximate nearest neighbor search with near-memory processing and hybrid early termination," in *ISCA*, 2025, pp. 1093–1107.

[6] L. Li, W. Sun, and B. Wu, "Dforest: A minimal dimensionality-aware indexing for high-dimensional exact similarity search," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 10, pp. 5092–5105, 2024.

[7] J. Wang, X. Yi, R. Guo, H. Jin, P. Xu, S. Li, X. Wang, X. Guo, C. Li, X. Xu, K. Yu, Y. Yuan, Y. Zou, J. Long, Y. Cai, Z. Li, Z. Zhang, Y. Mo, J. Gu, R. Jiang, Y. Wei, and C. Xie, "Milvus: A purpose-built vector data management system," in *SIGMOD*, 2021, pp. 2614–2627.

[8] "Weaviate." [Online]. Available: https://github.com/weaviate/weaviate

[9] "Qdrant." [Online]. Available: https://github.com/qdrant/qdrant

[10] "Source code and datasets in the benchmark." [Online]. Available: https://github.com/zzlin237/dco-benchmark.git

[11] G. Li, J. Sun, J. Pan, J. Wang, Y. Xie, R. Liu, and W. Nie, "Gaussdb-vector: A large-scale persistent real-time vector database for LLM applications," *PVLDB*, vol. 18, no. 12, pp. 4951–4963, 2025.

[12] Y. Han, C. Liu, and P. Wang, "A comprehensive survey on vector database: Storage and retrieval technique, challenge," *CoRR*, vol. abs/2310.11703, 2023.

[13] G. Casella and R. Berger, *Statistical inference*. Chapman and Hall/CRC, 2024.

[14] G. James, D. Witten, T. Hastie, and R. Tibshirani, *An introduction to statistical learning: with applications in R*. Springer, 2013, vol. 103.

[15] R. Szeliski, *Computer vision: algorithms and applications*. Springer Nature, 2022.

[16] C. Manning and H. Schutze, *Foundations of statistical natural language processing*. MIT press, 1999.

[17] B. Min, H. Ross, E. Sulem, A. P. B. Veyseh, T. H. Nguyen, O. Sainz, E. Agirre, I. Heintz, and D. Roth, "Recent advances in natural language processing via large pre-trained language models: A survey," *ACM Comput. Surv.*, vol. 56, no. 2, pp. 30:1–30:40, 2024.

[18] B. Pecher, I. Srba, and M. Bieliková, "A survey on stability of learning with limited labelled data and its sensitivity to the effects of randomness," *ACM Comput. Surv.*, vol. 57, no. 1, pp. 19:1–19:40, 2025.

[19] C. D. Toth, J. O'Rourke, and J. E. Goodman, *Handbook of discrete and computational geometry*. CRC press, 2017.

[20] M. Wang, X. Xu, Q. Yue, and Y. Wang, "A comprehensive survey and experimental comparison of graph-based approximate nearest neighbor search," *PVLDB*, vol. 14, no. 11, pp. 1964–1978, 2021.

[21] J. J. Pan, J. Wang, and G. Li, "Survey of vector database management systems," *VLDB J.*, vol. 33, no. 5, pp. 1591–1615, 2024.

[22] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with gpus," *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, 2021.

[23] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P. Mazaré, M. Lomeli, L. Hosseini, and H. Jégou, "The faiss library," *CoRR*, vol. abs/2401.08281, 2024.

[24] P. H. Chen, W. Chang, J. Jiang, H. Yu, I. S. Dhillon, and C. Hsieh, "FINGER: fast inference for graph-based approximate nearest neighbor search," in *WWW*, 2023, pp. 3225–3235.

[25] M. Muja and D. G. Lowe, "Fast approximate nearest neighbors with automatic algorithm configuration," in *VISAPP*, 2009, pp. 331–340.

[26] H. Abdi and L. J. Williams, "Principal component analysis," *WIRES COMPUT STAT*, vol. 2, no. 4, pp. 433–459, 2010.

[27] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824–836, 2020.

[28] A. Babenko and V. S. Lempitsky, "The inverted multi-index," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 37, no. 6, pp. 1247–1260, 2015.

[29] "Big ANN Benchmark," 2024. [Online]. Available: https://big-ann-benchmarks.com/

[30] "GloVe: Global Vectors for Word Representation." [Online]. Available: https://nlp.stanford.edu/projects/glove/

[31] C. Schuhmann, R. Vencu, R. Beaumont, R. Kaczmarczyk, C. Mullis, A. Katta, T. Coombes, J. Jitsev, and A. Komatsuzaki, "LAION-400M: open dataset of clip-filtered 400 million image-text pairs," *CoRR*, vol. abs/2111.02114, 2021.

[32] T. Nguyen, M. Rosenberg, X. Song, J. Gao, S. Tiwary, R. Majumder, and L. Deng, "MS MARCO: A human generated machine reading comprehension dataset," in *CoCo@NIPS*, vol. 1773, 2016.

[33] "New and improved embedding model," https://openai.com/index/new-and-improved-embedding-model/.

[34] "Distance comparison operations are not silver bullets in vector similarity search: An experimental study on their merits and limits (full paper)." [Online]. Available: https://github.com/zzlin237/dco-benchmark.git

[35] S. Jayaram Subramanya, F. Devvrit, H. V. Simhadri, R. Krishnawamy, and R. Kadekodi, "Diskann: Fast accurate billion-point nearest neighbor search on a single node," *NeurIPS*, vol. 32, 2019.

[36] H. Zhang, Y. Wang, Q. Chen, R. Chang, T. Zhang, Z. Miao, Y. Hou, Y. Ding, X. Miao, H. Wang, B. Pang, Y. Zhan, H. Sun, W. Deng, Q. Zhang, F. Yang, X. Xie, M. Yang, and B. Cui, "Model-enhanced vector index," in *NeurIPS*, 2023.

[37] P. Guide, "Intel® 64 and ia-32 architectures software developer's manual," *Volume 3B: system programming guide, Part*, vol. 2, no. 11, pp. 0–40, 2011.

[38] W. Li, Y. Zhang, Y. Sun, W. Wang, M. Li, W. Zhang, and X. Lin, "Approximate nearest neighbor search on high dimensional data - experiments, analyses, and improvement," *IEEE Trans. Knowl. Data Eng.*, vol. 32, no. 8, pp. 1475–1488, 2020.

[39] J. Qin, W. Wang, C. Xiao, and Y. Zhang, "Similarity query processing for high-dimensional data," *PVLDB*, vol. 13, no. 12, pp. 3437–3440, 2020.

[40] J. Qin, W. Wang, C. Xiao, Y. Zhang, and Y. Wang, "High-dimensional similarity query processing for data science," in *KDD*, 2021, pp. 4062–4063.

[41] K. Lu, M. Kudo, C. Xiao, and Y. Ishikawa, "HVS: hierarchical graph structure based on voronoi diagrams for solving approximate nearest neighbor search," *PVLDB*, vol. 15, no. 2, pp. 246–258, 2021.

[42] Z. Gong, Y. Zeng, and L. Chen, "Accelerating approximate nearest neighbor search in hierarchical graphs: Efficient level navigation with shortcuts," *PVLDB*, vol. 18, no. 10, pp. 3518–3530, 2025.

[43] J. Mohoney, A. Pacaci, S. R. Chowdhury, U. F. Minhas, J. Pound, C. Renggli, N. Reyhani, I. F. Ilyas, T. Rekatsinas, and S. Venkataraman, "Incremental ivf index maintenance for streaming vector search," *arXiv preprint arXiv:2411.00970*, 2024.

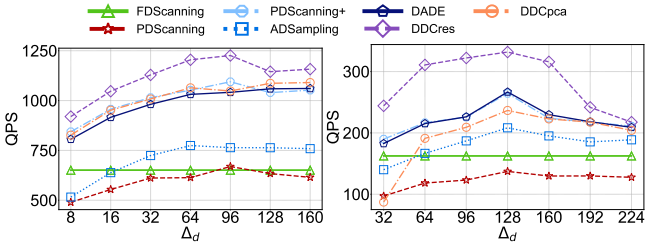
[44] B. Zheng, X. Zhao, L. Weng, Q. V. H. Nguyen, H. Liu, and C. S. Jensen, "PM-LSH: a fast and accurate in-memory framework for high-dimensional approximate NN and closest pair search," *VLDB J.*, vol. 31, no. 6, pp. 1339–1363, 2022.

[45] X. Zhao, Y. Tian, K. Huang, B. Zheng, and X. Zhou, "Towards efficient index construction and approximate nearest neighbor search in high-dimensional spaces," *PVLDB*, vol. 16, no. 8, pp. 1979–1991, 2023.

[46] Y. Tian, Z. Yue, R. Zhang, X. Zhao, B. Zheng, and X. Zhou, "Approximate nearest neighbor search in high dimensional vector databases: Current research and future directions," *IEEE Data Eng. Bull.*, vol. 47, no. 3, pp. 39–54, 2023.

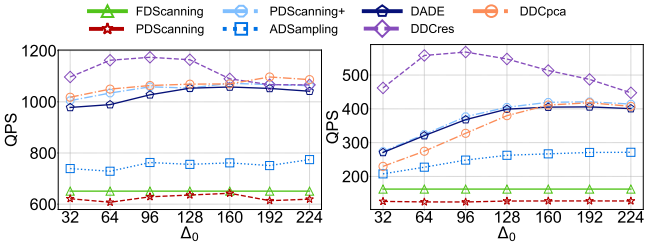
[47] L. Chen, Y. Gao, X. Song, Z. Li, Y. Zhu, X. Miao, and C. S. Jensen, "Indexing metric spaces for exact similarity search," *ACM Comput. Surv.*, vol. 55, no. 6, pp. 128:1–128:39, 2023.

[48] Y. Fu, C. Chen, X. Chen, W. Wong, and B. He, "Optimizing the number of clusters for billion-scale quantization-based nearest neighbor search," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 11, pp. 6786–6800, 2024.
[49] H. Jégou, M. Douze, and C. Schmid, "Product quantization for nearest neighbor search," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 33, no. 1, pp. 117–128, 2011.
[50] M. Wang, L. Lv, X. Xu, Y. Wang, Q. Yue, and J. Ni, "An efficient and robust framework for approximate nearest neighbor search with attribute constraint," in *NeurIPS*, 2023.
[51] A. A. Aomar, K. Echihabi, M. Arnaboldi, I. Alagiannis, D. Hilloulin, and M. Cherkaoui, "Rwalks: Random walks as attribute diffusers for filtered vector search," *Proc. ACM Manag. Data*, vol. 3, no. 3, pp. 212:1–212:26, 2025.
[52] W. Yang, T. Li, G. Fang, and H. Wei, "PASE: postgresql ultra-high-dimensional approximate nearest neighbor search extension," in *SIGMOD*, 2020, pp. 2241–2253.
[53] K. Figueroa, G. Navarro, and E. Chavez, "The metric spaces library maintained by the sisap initiative," 2017. [Online]. Available: https://github.com/kaarinita/metricSpaces
[54] "Hnswlib: fast approximate nearest neighbor search," 2025. [Online]. Available: https://github.com/nmslib/hnswlib



(a) HNSW (b) IVF

Fig. 16: Impact of incremental step size Δ_d



(a) HNSW (b) IVF

Fig. 15: Impact of initial step size Δ_0

APPENDIX A
DETAILED IMPLEMENTATION

We implemented all DCO algorithms from scratch within a unified framework, adhering to their open-source implementations [1]–[3]. This section will elaborate on the relevant implementation details.

Unified Framework. Every DCO algorithm was deployed within the same vector similarity search framework with the same index. During online query processing, all compared methods are implemented in C++, while Python is only used for pre-processing (*i.e.*, PCA and training), which is in line with existing work on DCOs [1]–[3].

Identical Index Configuration. In terms of indexes, we fix a consistent underlying data layout for all methods, where vectors in the IVF index are stored contiguously within each partition while those in the HSNW index follow the original insertion sequence, and none of the DCO methods are allowed to modify this predefined layout.

Batch Queries. Each query within a batch is processed sequentially and independently. We adopt this established evaluation protocol to maintain consistency with the baseline methods for a fair comparison.

Hardware Optimization. For HNSW, we utilize the SIMD-accelerated vector distance computation routines from the Hnswlib library [54], where a single SIMD instruction processes multiple dimension pairs simultaneously. On the GPU, we pre-load the IVF index into device memory and adopt fine-grained parallelism: a single CUDA kernel is launched per partition, with each thread processing one candidate vector. This configuration is identical across all DCO methods to ensure consistent execution patterns.

Consistent Implementation Languages. The online query processing of all compared methods is implemented in C++, while Python is only used for pre-processing (*i.e.*, PCA and training), which is in line with existing work on DCOs [1]–[3].

APPENDIX B
QUERY PROCESSING VIA HNSW ON CPUS WITH SIMD DISABLED

This experiment evaluates the performance improvement from applying DCOs to vector similarity search on HNSW with SIMD disabled.

Overall Query Performance without SIMD. Compared to the result in Fig. 4, the rankings of different DCO methods have undergone dramatic changes when SIMD is disabled. Specifically, FDScanning no longer achieves the best performance on the Deep dataset. Meanwhile, compared to FDScanning, hypothesis testing based methods improve QPS by up to 1.9–2.5×, while the classification based methods achieve 2.1–3.1× improvements. These advantages over FDScanning are larger than those observed when SIMD is enabled. This phenomenon can be attributed to the fact that SIMD primarily accelerates the distance computation itself, while the overhead of the termination decision logic, *i.e.*, determining whether to terminate computation remains unchanged. Consequently, the relative benefit of DCOs is diluted when SIMD is active, as the non-computational components dominate a larger fraction of the total time cost.

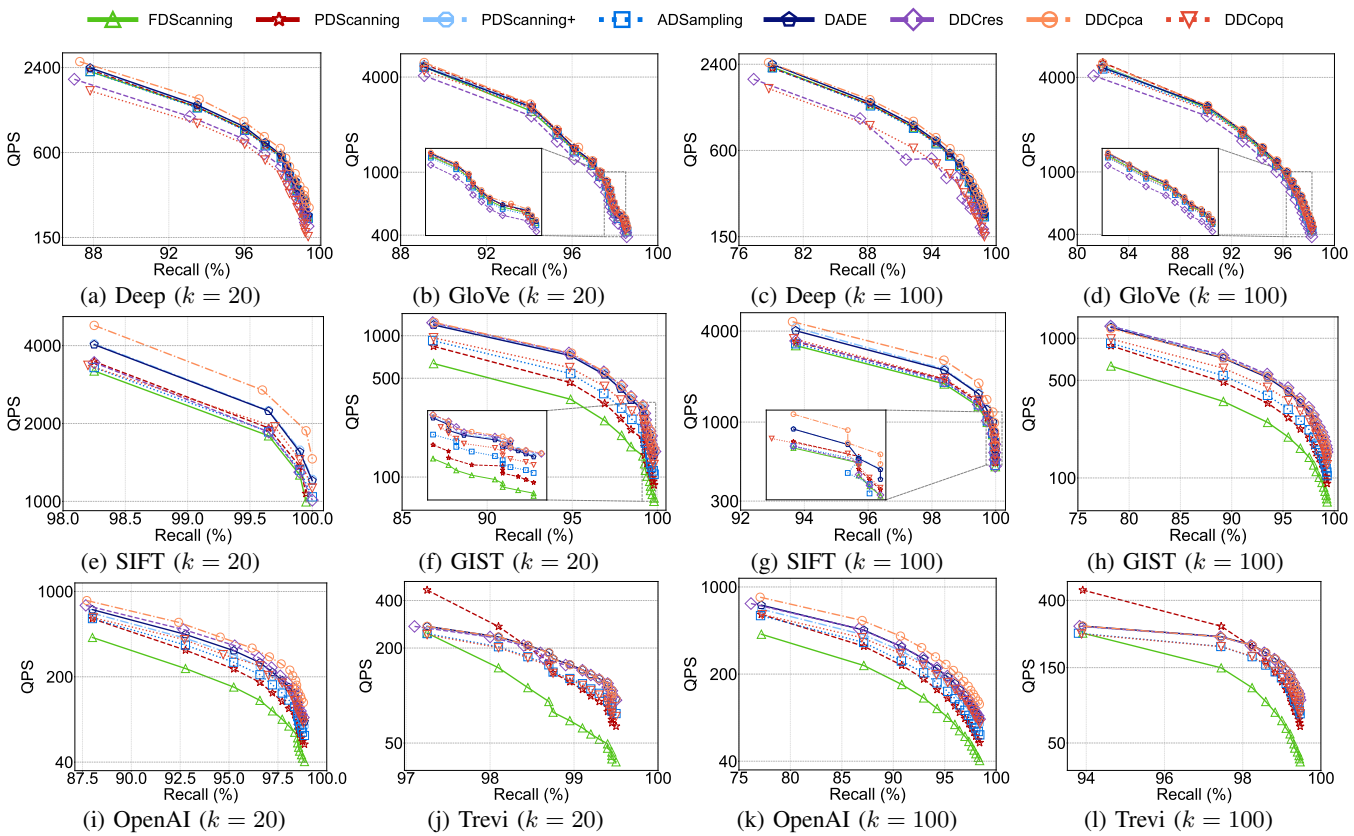
We also observe cases where the baseline PDScanning outperforms other methods. As shown in Fig. 17j and Fig. 17k, PDScanning achieves the highest QPS before the recall reaches 98%. This demonstrates that the online pre-processing time $O(N^2)$ is also a bottleneck for the SOTA methods for some ultra-high-dimensional datasets on HNSW when SIMD is disabled.

Takeaway: Enabling SIMD narrows the efficiency advantage of DCO methods over FDScanning, reducing the speedup from 1.9–3.1× to 1.1–2.1× on HNSW.

APPENDIX C
QUERY PROCESSING VIA HNSW WITH AVX2 SIMD INSTRUCTION SET

We evaluated the query performance of DCO methods using the AVX2 SIMD instruction set. As shown in Fig. 18, the overall performance trends are similar to those obtained with the SSE SIMD instruction set. For instance, on the low-dimensional GloVe dataset, most DCO methods fail to outperform FDScanning. On the ultra-high-dimensional Trevi dataset, both FDScanning and PDScanning maintain a performance lead until the recall rate exceeds 98%.

Since AVX2 is a more advanced instruction set than SSE, DCO methods (*e.g.*, FDScanning) typically run faster with AVX2 than with SSE. As a result, the relative speedup provided by DCOs diminishes when AVX2 is employed. Notably, with AVX2, FDScanning can surpass more recent SOTA methods. For example, FDScanning outperforms ADSampling on the GIST dataset in Fig. 18.



(a) Deep ($k = 20$) (b) GloVe ($k = 20$) (c) Deep ($k = 100$) (d) GloVe ($k = 100$)

(e) SIFT ($k = 20$) (f) GIST ($k = 20$) (g) SIFT ($k = 100$) (h) GIST ($k = 100$)

(i) OpenAI ($k = 20$) (j) Trevi ($k = 20$) (k) OpenAI ($k = 100$) (l) Trevi ($k = 100$)

Fig. 17: Query performance when DCOs are applied to vector similarity search on CPUs with SIMD disabled

Appendix D
Out-of-Distribution (OOD) Queries on CPUs with SIMD Disabled

To further evaluate the robustness of DCOs under out-of-distribution (OOD) queries, we also conduct this experiment with SIMD disabled.

As shown in Fig. 19a and Fig. 19c, DDCpca achieves the highest QPS on both datasets for in-distribution queries. However, it becomes the least efficient method under OOD queries, performing up to 1.6× and 1.8× slower than FDScanning and PDScanning, respectively. In this setting, aside from DDCopq, which achieves the highest QPS on the Laion dataset, the loop-based baselines PDScanning and PDScanning+ achieve the highest QPS on both datasets for recall over 97%. This aligns with our earlier finding: the SOTA methods often fail to maintain efficiency gains on OOD queries.

Appendix E
Experiment with Cosine Similarity

This experiment evaluates the extension of DCO methods to cosine similarity. To conduct this experiment, we select three datasets of varying dimensionality: GloVe, GIST, and Trevi.

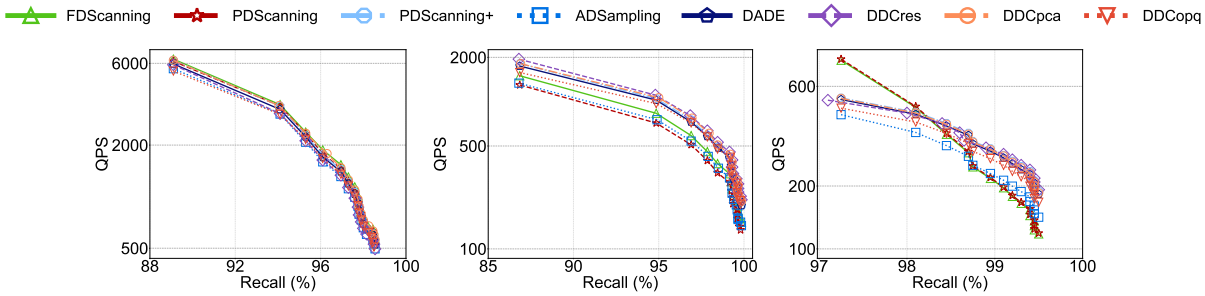
As illustrated in Fig. 20, the overall performance trend for cosine similarity is consistent with that observed for Euclidean distance and inner product. On the low-dimensional datasets (*e.g.*, GloVe), the efficiency advantage of the SOTA DCO methods is less pronounced, and they are even outperformed by FDScanning in some cases. For the ultra-high-dimensional dataset (*e.g.*, Trevi), the performance bottleneck caused by online pre-processing time persists consistently. This performance pattern stems primarily from two key factors: (1) most DCO extensions for cosine similarity are directly derived from their Euclidean distance equivalents, and (2) cosine similarity is mathematically equivalent to the inner product for normalized vectors.

Appendix F
Experiments of Diverse Hardware Configurations on XUltra Dataset

We further evaluate DCO methods across different hardware platforms on the XUltra dataset, which has the highest dimensionality among all evaluated datasets.

Performance on CPU with and without SIMD Support. As shown in Fig. 21a and Fig. 21b, PDScanning outperforms the SOTA methods due to the larger online pre-processing time before the recall reaches 89%. As the dimensionality further increases, DDCres attains the highest QPS, regardless of whether SIMD is enabled. Specifically, it improves QPS by up to 1.7× and 2.9× over FDScanning on CPUs with and without SIMD, respectively.

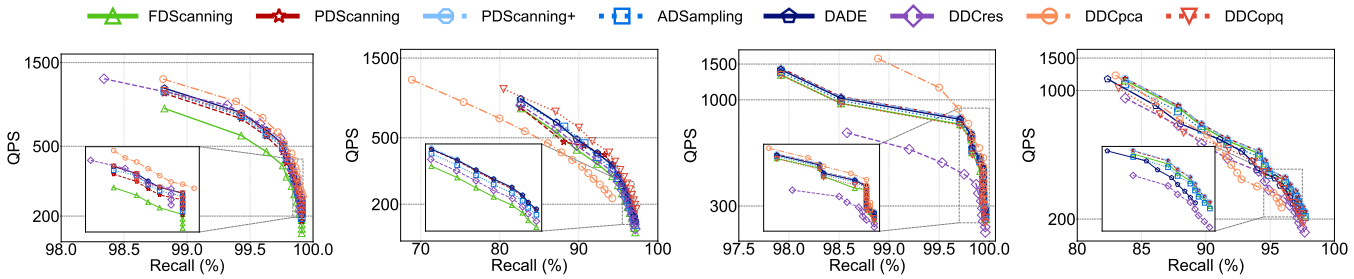
Performance on GPU. As shown in Fig. 21c, DDCres consistently achieves the highest QPS. Notably, the bottleneck



(a) GloVe, $k = 20$ (b) GIST, $k = 20$ (c) Trevi, $k = 20$

(d) GloVe, $k = 100$ (e) GIST, $k = 100$ (f) Trevi, $k = 100$

Fig. 18: Performance comparison on AVX2



(a) Laion (in-distribution) (b) Laion (OOD) (c) Text2Image (in-distribution) (d) Text2Image (OOD)

Fig. 19: Results on in-distribution and OOD queries without SIMD

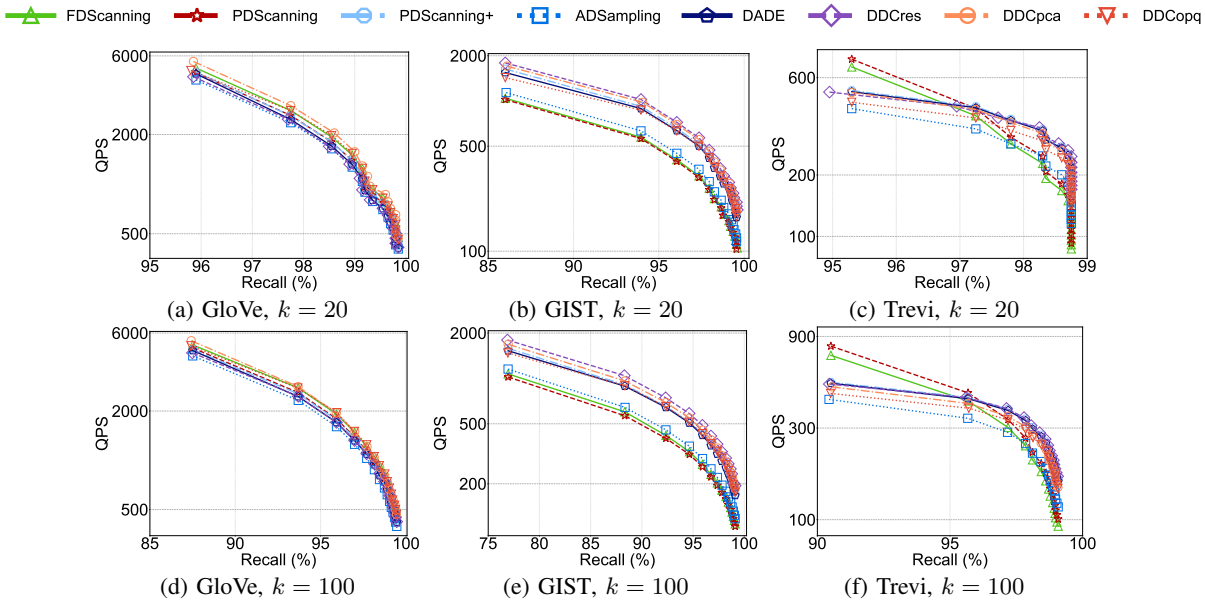
caused by online pre-processing time, observed on CPU, does not manifest on GPU. This can be attributed to the GPU's superior efficiency in matrix multiplication and parallel computation. The SOTA methods demonstrate strong performance on the ultra-high-dimensional XUltra dataset, improving QPS by up to 1.4–5.6× over FDScanning.

APPENDIX G
PRUNING CAPABILITY OF DCO UNDER THE IVF INDEX

We evaluate and analyze the capability of DCO on IVF. Fig. 22 illustrates the dimension pruning ratios of DCOs and their recall across three datasets of varying dimensionality.

Pruning Capability of DCO on IVF. The results also demonstrate that the pruning effectiveness of DCOs is dimension-dependent: higher pruning ratios are consistently observed on high-dimensional and ultra-high-dimensional datasets. On these datasets, DDCopq typically achieves the highest pruning ratio. However, it is worth noting that, in addition to dimension scanning, DDCopq incurs additional overhead from computing quantized distances. Consequently, despite its strong pruning capability, DDCopq exhibits poorer query performance on IVF when SIMD is disabled. When SIMD is enabled, thereby significantly accelerating quantized distance computation on IVF, the performance of DDCopq aligns much more closely with its pruning capability. For other DCO methods, a higher pruning capability generally translates to better acceleration when SIMD is disabled. Moreover, all methods maintain comparable recall, demonstrating that the minor approximation errors introduced by DCOs do not compromise query accuracy on IVF.

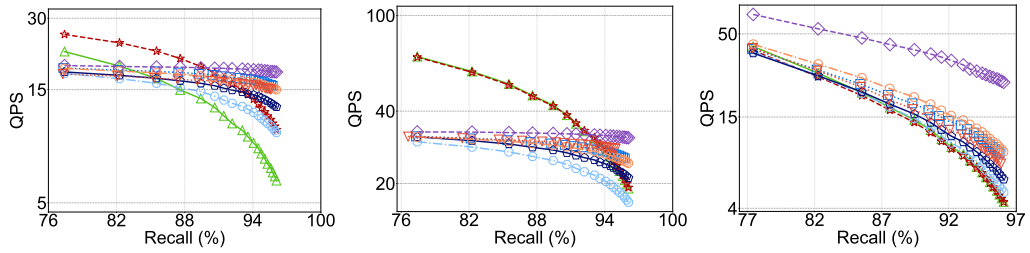
Pruning Capability Comparisons: HNSW vs IVF. Combined with the findings in Fig. 6, we observe that the pruning capability of DCOs remains relatively stable on HNSW, whereas it increases with $nprobe$ in IVF. This behavior can be attributed to the characteristics of candidate selection in each index structure. In HNSW, the retrieved candidates are typically close to the query vector. In contrast, IVF retrieves many candidates that are far from the query vector; for these, DCO methods can quickly determine irrelevance using only a few scanned dimensions. As a result, the pruning ratio in IVF rises with increasing $nprobe$.



(a) GloVe, $k = 20$ (b) GIST, $k = 20$ (c) Trevi, $k = 20$

(d) GloVe, $k = 100$ (e) GIST, $k = 100$ (f) Trevi, $k = 100$

Fig. 20: Search performance using cosine similarity



(a) XUltra (without SIMD) (b) XUltra (with SIMD) (c) XUltra (with GPU)

Fig. 21: Comparisons of DCO methods using the IVF index on XUltra dataset across different hardware environments

APPENDIX H
PARAMETER STUDY

This experiment investigates two primary parameters of DCOs: the *initial step size* Δ_0 (*i.e.*, the initial number of dimensions to scan), and the *incremental step size* Δ_d used for each subsequent scan. We systematically evaluate their effects on both HNSW and IVF indexes using the GIST dataset. Our study primarily reports QPS, as recall remains largely unaffected. Notice that, prior work [20], [35], [36] usually sets these parameters as $\Delta_d = \Delta_0 = 32$.

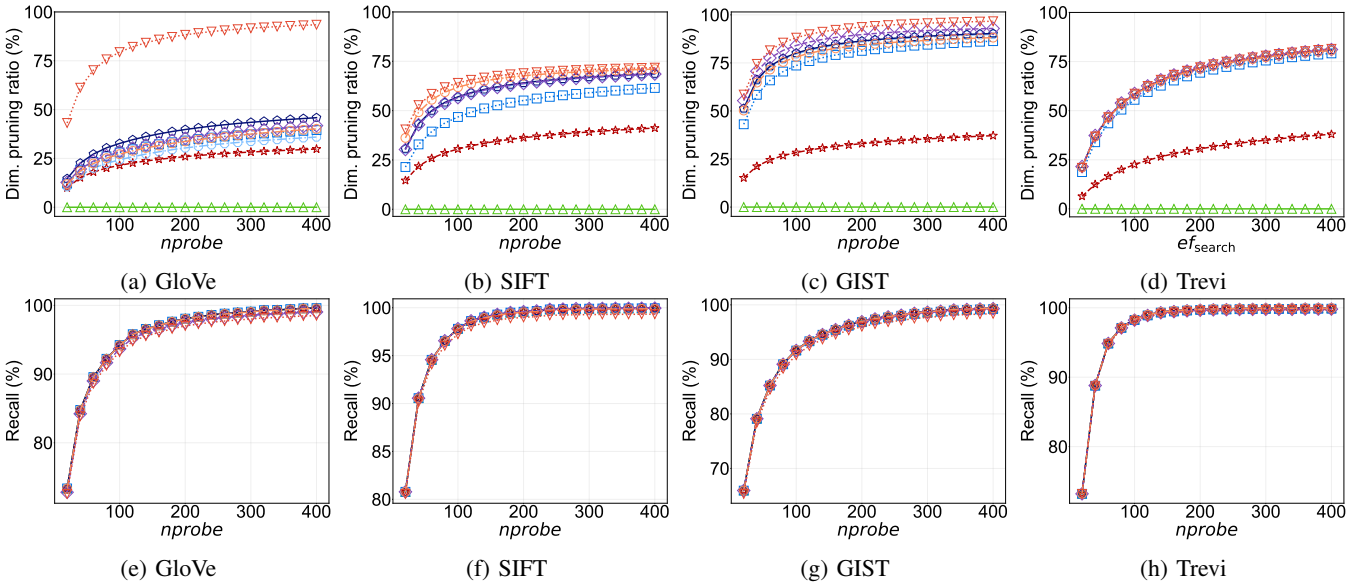
Impact of Initial Step Size Δ_0. Fig. 15 presents the QPS of each method on both HNSW and IVF indexes. In Fig. 15a, the QPS of simple scanning based methods remains relatively stable, whereas hypothesis testing based and classification based methods achieve a notable QPS increase by tuning Δ_0 appropriately. A similar trend is observed on the IVF index. ADSampling, DADE, DDCres, and DDCpca improve QPS by up to 1.3, 1.5, 1.2, and 1.8× respectively, in Fig. 15b.

The efficiency gain of tuning Δ_0 is more pronounced on IVF than on HNSW. This is because IVF stores each data partition contiguously in the main memory, a data layout that is cache-friendly. In contrast, the graph-based index HNSW lacks such a compact data layout. Consequently, even PDScanning+ achieves a considerable QPS improvement by adjusting Δ_0 in Fig. 15b.

Impact of Incremental Step Size Δ_d. Fig. 16 reports the QPS when varying Δ_d. Most algorithms are highly sensitive to this parameter, with FDScanning being the only exception. The optimal Δ_d is 64, 96 or 160 for HNSW, and 64 for IVF. Compared to the default $\Delta_d = 32$ in prior work [20], [35], [36], ADSampling, DADE, and DDCres achieve speedups of up to 1.1, 1.2, and 1.1×.

Overall, jointly using the optimal parameter combinations, ADSampling, DADE, and DDCres can boost QPS by up to 1.5, 1.8, and 1.8× compared to the settings in prior work.

Takeaway: While tuning Δ_0 and Δ_d can substantially boost QPS, their optimal values are determined by datasets, indexes, and specific DCO methods.



(a) GloVe (b) SIFT (c) GIST (d) Trevi

(e) GloVe (f) SIFT (g) GIST (h) Trevi

Fig. 22: Dimension pruning via DCO and its impact on recall on IVF